Tunisian Republic

Ministry of Higher Education and Scientific Research

Private Higher School of Engineering and Technology

TEK-UP

Final Year Project Report

Submitted in partial fulfilment of the requirements for the

National Engineering Degree in Applied and Technological Sciences

Speciality: Web, Mobile and Multimedia Development

Prepared by

Nadim JEBALI

Design and Development of an Intelligent Multi-Vendor Marketplace: Cheebo

Professional Supervisor: Mr. Mohammed Aziz Chibani
NeuralBey

Academic Supervisor: Ms. Sawssen Jalel
Lecturer

Academic Year 2025 – 2026

I authorise the student to submit his internship report for a defence.

Professional Supervisor, Mr. Mohammed Aziz Chibani

Signature and stamp

I authorise the student to submit his internship report for a defence.

Academic Supervisor, Ms. Sawssen Jalel

Signature

Dedication

To my parents, for their unwavering love, patience and encouragement during every step of this project.

To my friends and colleagues at NeuralBey and TEK-UP who offered guidance, feedback and inspiration.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this project.

First, my deepest thanks to my professional supervisor, Mr. Mohammed Aziz Chibani (NeuralBey), for his continuous guidance, domain expertise, and support throughout the development and evaluation phases.

I am also grateful to Ms. Sawssen Jalel for her academic supervision and valuable feedback which helped shape the report.

Special thanks to my colleagues and friends for testing features, providing constructive critiques, and motivating me during challenging moments.

Finally, I thank my family for their encouragement and patience during this work.

Abstract

Keywords: marketplace, multi-vendor, NestJS, React, Prisma, Artificial Intelligence, Docker, CI/CD, Terraform.

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CD	Continuous Deployment
CI	Continuous Integration
DB	Database
DNS	Domain Name System
DTO	Data Transfer Object
DWMM	Web, Mobile and Multimedia Development
JWT	JSON Web Token
ORM	Object-Relational Mapping
PFE	Final Year Project
REST	Representational State Transfer
SPA	Single Page Application
SSH	Secure Shell
UML	Unified Modeling Language
VPC	Virtual Private Cloud

General Introduction

This introduction explains the context for the Cheebo project, why the system was created, who the intended users are, and how the work was organised from conception through delivery.

Cheebo is a multi-vendor marketplace developed as a final year engineering project in partnership with TEK-UP and NeuralBey. The platform is designed to give Tunisian sellers a modern, secure online storefront while offering customers an intelligent shopping experience and administrators a reliable way to manage vendors, products, and verification workflows.

The project combines academic goals and real-world needs: it must satisfy the PFE requirements for software engineering, while also providing a practical solution that addresses the host organisation's expectations for a modern web application. The following chapters present the host organisation, the business context, the requirements analysis, the system architecture, implementation details, and the deployment strategy.

Chapter 1 General Project Context

This chapter sets the scene — why this project exists, who it is for, and how its development was organised. It introduces the host organisation, the business motivation behind Cheebo, the competitive landscape in which it operates, the proposed solution, and the working methodology adopted throughout the internship.

1.1 Presentation of the Host Organisation

1.1.1 NeuralBey — Company Overview

NeuralBey is a Tunisian technology company that delivers end-to-end engineering services across a broad spectrum of modern computing fields. The company positions itself as a partner that translates client visions into working systems, summarised by its own statement: *"From web and mobile apps to AI systems, IoT, embedded engineering, and 3D solutions — we build the technology that powers your vision."*

This breadth allows NeuralBey to handle a project from initial concept through deployment without external dependencies, and to combine disciplines that are usually siloed — for example, integrating artificial intelligence into conventional web platforms, or interfacing embedded devices with cloud services. The company's portfolio reflects this versatility:

- **Web and mobile applications** for consumer products and enterprise tooling;
- **AI systems**, including model integration, automation pipelines and intelligent assistants;
- **IoT and embedded engineering**, where NeuralBey designs firmware and connected devices;
- **3D solutions** for product visualisation, AR/VR and digital twins.

1.1.2 Domain, Team and Mission

NeuralBey operates as a multidisciplinary studio. Each project is staffed by engineers selected from the relevant domain (frontend, backend, AI, hardware, 3D), and a professional supervisor accompanies the work from scoping through delivery. The team's working culture emphasises code review, iterative delivery and frequent stakeholder

feedback — practices that aligned naturally with the academic requirements of a final year project.

The present project was carried out under the professional supervision of **Mr. Mohammed Aziz Chibani**, who provided architectural guidance, regular reviews, and technology recommendations throughout the internship. Academic oversight was provided in parallel by **Ms. Sawssen Jalel**, ensuring the work also satisfied the methodological and documentation standards of the engineering diploma.

1.2 Project Context

1.2.1 Project Origin and Motivation

The Cheebo project originated from an internal need identified at NeuralBey: the Tunisian online retail landscape lacks a modern, multi-vendor platform that is both seller-friendly and equipped with the automation features expected of contemporary marketplaces. Existing local solutions are typically single-vendor stores or classified-ad sites, and they offer limited tooling for moderation, content quality and discovery.

NeuralBey saw an opportunity for a product combining three differentiating capabilities:

(i) a true multi-vendor model with self-service onboarding for sellers;

(ii) AI-assisted moderation of stores and products to reduce manual review load;

(iii) a deployment story that demonstrates contemporary DevOps practices end-to-end.

The PFE topic was therefore scoped jointly with the professional supervisor, with the explicit goal of producing a system that could serve both as a portfolio piece for the company and as a foundation for further internal development beyond the internship.

1.2.2 Multi-Vendor Marketplace Concept

A *multi-vendor marketplace* is a platform on which independent sellers operate their own stores within a shared environment governed by a central operator. It differs from two adjacent models:

- a **single-vendor store**, where the platform itself owns the inventory and the brand;
- a **classified-ad site**, where the platform only lists offers and plays no role in transactions or quality control.

A marketplace must reconcile three concerns simultaneously: **seller autonomy** (each seller manages catalogue, branding and inventory), **buyer trust** (the platform

guarantees a baseline of product authenticity and fulfilment) and **operator governance** (administrators onboard sellers, moderate content, suspend stores and curate featured selections). These three perspectives drive the architecture of Cheebo: every feature is designed to balance autonomy for sellers, trust for buyers, and control for administrators. The integration of AI verification is a direct response to the buyer-trust requirement at the scale a marketplace demands.

1.2.3 Target Users

Three actor categories were identified at the scoping stage:

- **Customer** — an end-user who browses the catalogue, places orders and tracks deliveries. Expects a fast, mobile-friendly experience with reliable search and clear product information.
- **Seller** — a small or medium merchant who creates a store, manages a product catalogue and fulfils orders. Expects low onboarding friction, clear analytics, and protection against unfair competition from low-quality listings.
- **Administrator** — the platform operator (NeuralBey staff) who reviews seller applications, moderates flagged content, curates featured stores, and intervenes when a store violates the marketplace rules.

1.3 Study of Existing Solutions

1.3.1 Existing Tunisian E-Commerce Platforms

Three categories of competing platforms were studied:

- **Pan-African marketplaces (Jumia).** Jumia Tunisia is the most visible regional marketplace. It supports multiple vendors at scale and offers a polished customer experience, but seller onboarding is heavyweight and the platform is not tailored to small Tunisian merchants.
- **Classified-ad sites (Tayara, Affariyet).** Tayara dominates the local classified-ad market. It offers excellent discovery for sellers, but performs no escrow, no order management, no integrated payment and no quality moderation.
- **Single-vendor stores (MyTek, Wiki, etc.).** These are vertically integrated retailers that operate their own catalogue. They cannot be used by independent sellers.

1.3.2 Limitations of Current Solutions

From the study above, four gaps emerged:

L1. No AI-assisted moderation. On existing platforms, store and product verification are entirely manual — slow, inconsistent, and impractical at scale.

L2. Heavyweight seller onboarding. Pan-regional players require contractual paperwork unsuitable for small merchants.

L3. Limited operator tooling. Classified-ad sites have weak admin dashboards; single-vendor stores have none for external sellers.

L4. Outdated technical foundations. Several platforms still rely on monolithic stacks without modern CI/CD or infrastructure-as-code, making evolution costly.

1.3.3 Competitive Analysis

Table 1.1 summarises the comparison along the four criteria that drove the design of Cheebo. A check mark (✓) indicates the criterion is fully supported, a cross (✗) indicates it is absent, and a tilde (∼) indicates partial support.

Table 1.1: Comparative analysis of competing platforms

Criterion	Jumia	Tayara	Cheebo
Multi-vendor model	✓	∼	✓
AI verification	✗	✗	✓
Admin dashboard	∼	✗	✓
CI/CD pipeline (open)	✗	✗	✓
Lightweight onboarding	✗	✓	✓

The `C` column type used above is defined locally as a centred variant of `X`; it is declared once in the preamble.

1.4 Proposed Solution

1.4.1 Cheebo Overview

Cheebo is a multi-vendor marketplace designed around three pillars: a self-service seller experience, AI-assisted content verification, and a modern, fully containerised deployment. The platform is built as a single-page **React 19** application backed by a modular **NestJS** API, with **MariaDB** as the relational store and **n8n** acting as an orchestration layer for AI tasks. Authentication is handled by **Better-Auth** with email verification, and **Prisma** is used as the ORM. The full system is packaged with Docker Compose, provisioned on DigitalOcean via Terraform, and delivered through a GitHub Actions CI/CD pipeline.

Sellers create a store in minutes, populate it with products and tags, and benefit from automatic AI verification that surfaces a trust badge to potential buyers. Administrators review borderline cases through a dedicated dashboard, suspend stores when needed, and override automated decisions, and query the database through an in-app AI chat assistant for operational insights. Customers browse a unified catalogue and place orders across multiple sellers in a single checkout.

1.4.2 Key Differentiators

Seven differentiators set Cheebo apart from the platforms surveyed in the previous section:

D1. AI verification of stores and products via an n8n workflow that calls a hosted LLM and writes back the `aiReviewed` flag plus a textual `aiReviewNote` rationale.

D2. Lightweight seller onboarding through a seller application form reviewed by administrators, with a free tier offering a single store at zero cost.

D3. Integrated administration tooling (review, suspend, curate, cascade-delete) implemented as first-class features rather than database admin scripts.

D4. Conversational data access via a database-aware chatbot built on n8n, allowing administrators to query the platform in natural language.

D5. AI-assisted storefront redesign — sellers can regenerate their entire store visual identity from a natural-language prompt, powered by an n8n workflow and the Groq `llama-3.3-70b-versatile` model.

D6. Store-wide sale promotions that propagate a percentage discount automatically across all product cards, store listings, and search results, and apply the discounted price at checkout without any per-product editing.

D7. Modern DevOps foundations — pnpm monorepo, Docker Compose, Terraform on DigitalOcean, and GitHub Actions CI/CD — demonstrating production-grade engineering practices end-to-end.

1.4.3 Project Scope

The scope was agreed with the professional supervisor and is split between in-scope and out-of-scope items so that the deliverables remain clear and bounded.

In scope: account creation and email verification; seller application workflow with admin review; store and product CRUD with image upload; tag-based product recommendations with a top-seller fallback; order placement, items and history; admin moderation panels (store suspension, cascade-delete); AI verification and chat assistant via n8n; store-wide percentage sales with automatic badge propagation across products

and search results; AI-assisted storefront redesign via an n8n/Groq workflow; premium tier unlocking additional stores; and infrastructure-as-code deployment.

Out of scope (deferred to future iterations): integrated payment processing (e.g. Konnect or Stripe); a native mobile application; advanced fraud detection; on-premise LLM hosting; multi-currency support; and a full-text search engine such as Elasticsearch or Typesense.

1.5 Work Methodology

1.5.1 Classical vs Agile Approaches

Every software project requires a methodological framework to organise work, manage priorities, and guarantee the delivery of a quality product. Two major families of methods have historically competed: the **classical** (predictive) approach and the **agile** (adaptive) approach. Their fundamental difference lies in how they handle change and value delivery.

Classical approach — Waterfall. The Waterfall method divides the project into strictly sequential phases: requirements, design, development, testing, and delivery. Each phase must be fully validated before the next begins. This model suits projects whose requirements are stable and known in advance, but it shows its limits as soon as the context evolves:

- Requirements are frozen at launch; any late change is costly.
- The client sees a working product only at the very end of the project.
- Defects and risks are detected in the test phase, too late to address without significant impact.
- Documentation tends to be voluminous and quickly out of sync with the actual code.

Agile approach — Core principles. Agility was born from the *Agile Manifesto* (2001), which champions collaboration, adaptability, and the continuous delivery of value. Rather than planning the entire project upfront, work is organised into **short cycles called sprints** (one to four weeks), each producing a functional and tested increment. This approach offers several concrete advantages:

- Requirements are refined continuously from client and stakeholder feedback.
- Value is delivered from the first sprints; the product is usable well before the end of the project.

- Risks are identified and addressed early.
- Daily communication fosters responsiveness and team alignment.

1.5.2 Why Scrum?

Among the agile methodologies available — Scrum, Kanban, XP, SAFe — **Scrum** was chosen to manage this project for the following reasons:

- **Flexibility:** the requirements of Cheebo evolved throughout the internship; Scrum allows supervisor feedback to be integrated at each sprint.
- **Visibility:** a prioritised backlog and bounded sprints provide a clear view of progress at any point.
- **Regular rhythm:** two-week sprints impose a structuring cadence well-suited to the internship calendar.
- **Frequent deliveries:** each sprint produces a functional increment, facilitating demonstrations and validations with the professional supervisor.
- **Industry adoption:** Scrum is the most widely practised agile framework, and its use in a PFE context prepares the student for professional practice.

1.5.3 The Scrum Framework

Scrum is an agile project management framework that helps teams organise and supervise their work through values, principles, and practices. It encourages teams to learn from experience, self-organise while working on a problem, and reflect on their successes and failures to continuously improve.

Figure 1.1 illustrates the Scrum process: successive sprints feed functional increments into the product, validated at each iteration by the Product Owner.

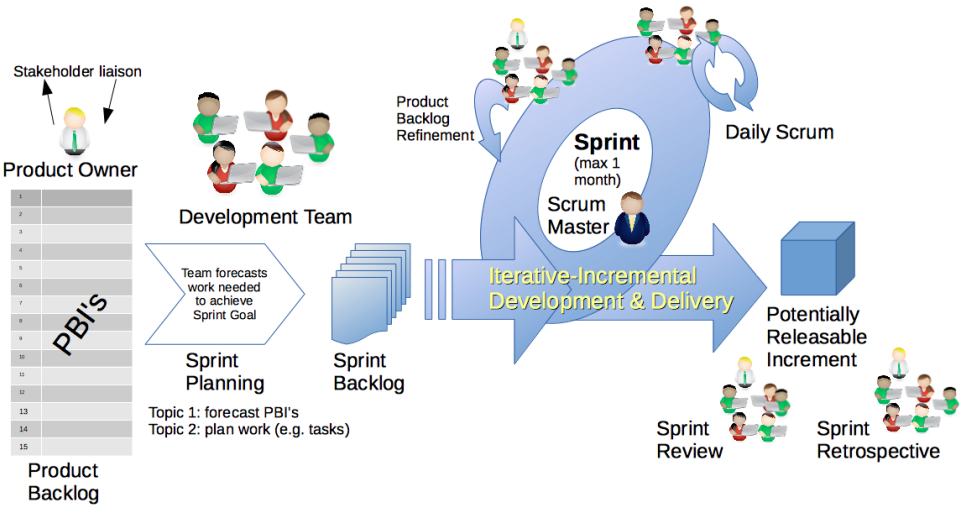

Figure 1.1: Scrum methodological framework (source: Dr Ian Mitchell, Wikimedia Commons, CC0 / CC BY-SA 4.0).

1.5.4 Scrum Roles

Scrum defines three complementary roles:

- **Product Owner:** represents the stakeholders and defines the product vision. He prioritises the Product Backlog items and validates each delivered increment. In this project, this role was fulfilled by **Mr. Mohammed Aziz Chibani** (NeuralBey), who approved the sprint backlog and conducted sprint reviews.
- **Scrum Master:** guarantees the application of the Scrum framework, facilitates team work, and removes obstacles. Given the single-developer nature of this PFE, this role was also assumed by Mr. Chibani, who provided architectural guidance and methodology coaching.
- **Development Team:** responsible for the technical realisation of features and the delivery of a potentially deployable increment at the end of each sprint. Composed in this project of the student, **Nadim Jebali**.

1.5.5 Scrum Events

The Scrum framework organises work around five key events:

- **Sprint:** a fixed-duration iteration (two weeks in this project) during which the team produces a product increment.
- **Sprint Planning:** an opening meeting in which the team selects and plans the backlog items to realise during the sprint.

- **Daily Scrum:** a brief daily synchronisation of 15 minutes to coordinate work and surface blockers.
- **Sprint Review:** a demonstration of the increment to stakeholders at the end of the sprint to collect feedback and validate progress.
- **Sprint Retrospective:** a process improvement meeting at the end of each sprint. Key improvements that emerged from retrospectives in this project include the adoption of `pnpm` for monorepo management, the migration from manual deployments to GitHub Actions, and the introduction of n8n for AI workflows.

1.5.6 Scrum Artefacts

Three artefacts ensure transparency of the work:

- **Product Backlog:** a prioritised list of all expected product functionalities, maintained as user stories with MoSCoW priorities and complexity estimates (see Chapter 2).
- **Sprint Backlog:** a sub-set of the Product Backlog selected for a given sprint, broken down into technical tasks.
- **Increment:** a potentially deliverable version of the product obtained at the end of each sprint. The full project ran across approximately ten sprints, grouped into three milestones:
- **Foundations (sprints 1–3):** repository setup, authentication with Better-Auth, base data model, and the seller-application flow.
- **Core marketplace (sprints 4–7):** stores, products, orders, admin panels, and tag-based recommendations.
- **Intelligence and delivery (sprints 8–10):** the AI verification pipeline, the in-app AI chat assistant, Dockerisation, Terraform provisioning, and the CI/CD pipeline.

1.5.7 Tools Used

The following tools were used throughout the project, covering source control, development, containerisation, infrastructure provisioning, workflow automation, and API testing.

GitHub

`github.com`

A web-based platform for version control and collaborative software development built on Git. It was used for source control, code review via pull requests, and continuous integration and deployment through GitHub Actions workflows.

Notion

`notion.so`

An all-in-one productivity workspace combining notes, databases, and kanban boards. It served as the project's backlog management tool, sprint board, and a living record of user stories and acceptance criteria.

Visual Studio Code

`code.visualstudio.com`

A lightweight but powerful source-code editor developed by Microsoft. It was the primary IDE throughout the project, extended with plugins for TypeScript, Prisma schema editing, ESLint linting, and Git integration.

Docker

`docker.com`

An open platform for developing, shipping, and running applications inside isolated containers. It was used to containerise all services — MariaDB, the NestJS backend, the React frontend, and the n8n workflow engine — ensuring parity between development and production environments.

Terraform

`terraform.io`

An open-source infrastructure-as-code tool by HashiCorp that allows cloud resources to be defined in declarative configuration files. It was used to provision and manage the DigitalOcean droplet, DNS records, and firewall rules in a reproducible way.

n8n

`n8n.io`

A fair-code workflow automation platform with a visual node-based editor. It was used to orchestrate the AI-assisted product and store verification pipeline as well as the in-app AI chat assistant, connecting the backend to large language model APIs.

Postman

postman.com

A collaborative API development platform for designing, testing, and documenting HTTP endpoints. It was used during backend development to manually exercise and validate every REST endpoint before integration with the frontend.

pnpm

pnpm.io

A fast, disk-efficient package manager for Node.js that uses hard links and a content-addressable store. It managed all JavaScript dependencies across the monorepo workspace, ensuring consistent installs in both development and CI.

Key Takeaways

- Cheebo originates from an internal NeuralBey brief: build a modern, Tunisia-oriented multi-vendor marketplace with AI-assisted moderation.
- Three actors — customer, seller, administrator — drive every architectural decision throughout the report.
- Compared with Jumia and Tayara, Cheebo differentiates on *AI verification*, *lightweight seller onboarding*, *first-class admin tooling*, and *modern DevOps* (Docker, Terraform, GitHub Actions).
- Development followed two-week Scrum sprints with NeuralBey reviews, structured around three milestones: foundations, core marketplace, and intelligence-and-delivery.

Chapter Conclusion

This chapter established the foundations of the Cheebo project. It introduced NeuralBey, the host organisation, and explained the business motivation behind building a modern Tunisian multi-vendor marketplace. A study of existing platforms revealed four gaps — absent AI moderation, heavyweight onboarding, weak admin tooling, and outdated technical foundations — that Cheebo directly addresses. The proposed solution, its key differentiators, and its bounded scope were presented. Finally, the Scrum agile methodology was described in detail, including its roles, events, and artefacts, and the three-milestone sprint plan that organised the ten-week development was outlined. The following chapter formalises the requirements derived from this context.

Chapter 2 Requirements Analysis and Specification

This chapter identifies the three system actors, formalises the functional and non-functional requirements of Cheebo, and illustrates them with UML use-case and sequence diagrams. The requirements were consolidated during the first three sprints in collaboration with the professional supervisor and serve as the contract that the rest of the report builds upon.

2.1 Identification of Actors

The platform recognises three actor categories. They are distinct *roles* stored on the `User` table rather than distinct user records — a single account can be promoted from *customer* to *seller* once a seller application is approved, and an administrator account is created automatically on first boot.

2.1.1 Customer

The Customer is the default role assigned upon registration. A Customer can browse the catalogue, search products, view individual stores, manage a cart, place an order across multiple sellers in a single checkout, and review their personal order history. The Customer is also the entry point for the seller pipeline: any Customer can submit a seller application and, once approved, gain the Seller role on the same account. Anonymous (non-authenticated) visitors share the read-only abilities of the Customer but cannot maintain a cart or place an order.

2.1.2 Seller

The Seller is a Customer whose seller application has been approved by an administrator. A Seller can create and edit one store under the free tier (or several under the premium tier), populate that store with products, upload product images, attach product tags, view per-store analytics, and consult the order history specific to their store. Sellers also choose a visual template for their store from the templates exposed by the frontend. They cannot, however, review or moderate the work of other sellers.

2.1.3 Administrator

The Administrator is created automatically by the backend at first boot and is not exposed through the registration form. The Administrator reviews seller applications, browses every store and every product on the platform, suspends or deletes stores (with cascading deletion of dependent records), overrides automated AI verification decisions, curates featured stores and products on the landing page, manages categories and subcategories, and consults the platform through a database-aware AI chat assistant.

2.2 Functional Requirements

The functional requirements are grouped into seven domains. Each domain corresponds to a NestJS module on the backend and to a matching set of pages on the React frontend; the mapping is made explicit in Chapter 3.

2.2.1 User Registration and Authentication

FR-A1. The system shall allow a visitor to create an account with an email address and password.

FR-A2. The system shall send a verification email containing a one-time link upon successful registration.

FR-A3. The system shall prevent unverified accounts from creating a store or placing an order.

FR-A4. The system shall authenticate returning users via Better-Auth sessions and issue an HTTP-only cookie.

FR-A5. The system shall allow the user to log out and invalidate the corresponding session.

FR-A6. The system shall enforce role-based access control on every non-public route through guards and the `@Roles()` decorator.

2.2.2 Store Management

FR-S1. A Seller shall be able to create a store with a name, description, logo and template.

FR-S2. A Seller shall be able to edit their store profile at any time.

FR-S3. A free-tier Seller shall be limited to a single store.

FR-S4. A premium-tier Seller shall be able to operate multiple stores from the same account.

FR-S5. Each store shall expose a public storefront URL identified by a human-readable slug derived from the store name (e.g. `/stores/pawsome-pet-foods`), rather than by its numeric primary key.

FR-S6. Slug uniqueness shall be enforced at the database level and collisions resolved by an automatic numeric suffix (`-2`, `-3`, ...). The slug shall be regenerated when the store name changes.

FR-S7. The store status (active, pending, suspended) shall be persisted in a `StoreStatusLog` audit trail.

FR-S8. The Seller shall be able to customise the storefront across seven independent sections — *Theme*, *Announcement Bar*, *Banner*, *Header*, *Product Grid*, *Sidebar* and *Footer* — through a WYSIWYG visual editor built with Craft.js, where clicking any section on the live canvas immediately opens its settings panel.

FR-S9. Four pre-built presets shall be available as starting points for customisation: *Default* and *Minimal* are available to all sellers, while *Modern* and *Classic* are restricted to premium accounts. Applying a preset overwrites the current draft with the preset's section defaults.

FR-S10. The Seller shall be able to upload a custom banner image, stored under `/uploads/stores/` and referenced from the customisation JSON.

FR-S11. The customisation editor shall support undo/redo on every section change and shall warn the seller before discarding unsaved changes.

FR-S12. Customisation features shall be tiered. Free accounts may use a curated subset — a six-colour palette for the primary colour, two fonts (*Inter* and the system default), the *soft* corner radius, checkbox-style filters, square product images, and section visibility toggles. Premium accounts unlock the full set: free-form hex colours, all four fonts, all radii, banner image upload with heading and overlay controls, sidebar inner panel and text colours, non-checkbox filter styles, product grid background, non-square aspect ratios, and footer social links. When a premium subscription expires, the seller's existing customisation values are preserved at the database level and remain visible to visitors; the seller is only prevented from *modifying* those premium fields going forward.

FR-S13. A Seller shall be able to activate a store-wide sale by enabling a percentage discount (1–99 %), an optional custom label (e.g. *"Summer Sale"*), and an optional expiry date. While the sale is active, every product in the store shall display its discounted price alongside a red sale badge on product cards, the

store card, and search results. The discounted price shall also be used as the unit price when the product is added to an order.

FR-S14. A Seller shall be able to trigger an AI-assisted full storefront redesign by submitting a short natural-language prompt. The system shall forward the request to an n8n workflow powered by the Groq `llama-3.3-70b-versatile` model, which generates a complete `StoreCustomization` object covering all seven sections. The result shall be applied to the live Craft.js editor immediately, allowing the seller to review or further refine the design before saving.

2.2.3 Product Management

FR-P1. A Seller shall be able to create, edit and delete products belonging to their own store(s).

FR-P2. Each product shall accept one or more images of supported MIME types, with a configurable maximum file size.

FR-P3. Each product shall support an arbitrary number of tags used by the recommendation engine.

FR-P4. Each product shall carry two review flags: `aiReviewed`, set automatically by the AI verification pipeline, and `adminReviewed`, set when an Administrator manually overrides the AI verdict.

FR-P5. Products shall be browsable by category and sub-category.

2.2.4 Order System

FR-O1. A Customer shall be able to add and remove items in a cart persisted on the client side.

FR-O2. Checkout shall create a single `Order` entity even when the cart spans multiple stores, with one `OrderItem` per product line.

FR-O3. Order placement shall trigger a confirmation email through Nodemailer.

FR-O4. Each Customer shall be able to consult their personal order history.

FR-O5. Each Seller shall be able to consult the orders that include at least one product from their store.

2.2.5 Admin Dashboard

FR-D1. The Administrator shall access a dashboard listing pending seller applications, recent activity and platform-wide statistics.

FR-D2. The Administrator shall be able to approve or reject seller applications, granting the Seller role on approval.

FR-D3. The Administrator shall be able to suspend, reactivate or delete any store, with cascading deletion of its products and orders.

FR-D4. The Administrator shall be able to override the `adminReviewed` verdict of any product or store.

2.2.6 AI Verification

FR-V1. Upon creation, every store and product shall be sent to an n8n workflow (`cheebo-ai-verify`) that calls a hosted large-language model.

FR-V2. The workflow shall return a decision (*approve*, *flag*, *reject*) together with a textual rationale, written back to the entity through a callback endpoint.

FR-V3. Flagged entities shall be listed in the moderation queue for administrative review.

2.2.7 Featured Content Curation

FR-C1. The Administrator shall be able to mark any store or product as *featured*.

FR-C2. Featured entities shall be promoted on the landing page.

FR-C3. In the absence of featured products for a given tag, the recommendation engine shall fall back to top-selling products of that tag.

2.3 Non-Functional Requirements

The non-functional requirements, summarised in Table 2.1, were prioritised by the professional supervisor during the requirements review and define the qualities the deliverable must exhibit beyond pure functionality.

Table 2.1: Non-functional requirements

Criterion	**Description**
Performance	The catalogue and store pages must render in under one second on a desktop connection of 10 Mbit/s; API endpoints under the median use case must respond in under 300 ms.
Security	All passwords are hashed by Better-Auth; sessions are stored in HTTP-only, `SameSite=Lax` cookies; uploads are MIME- and size-validated; every protected route is gated by an `AuthGuard` and the `@Roles()` decorator.
Scalability	The backend is stateless and horizontally scalable behind a load balancer; uploads are written to a Docker volume that can be migrated to object storage with no code change.
Availability	The production stack runs under Docker Compose with restart policies; CI/CD deployment is zero-downtime through rolling container replacement.
Usability	The frontend follows the accessible component patterns of Radix UI, is fully responsive, and complies with the WCAG 2.1 AA contrast guidelines for primary text. Public URLs are slug-based and human-readable, improving share-ability and search-engine indexing.
Maintainability	The codebase is split into clearly-bounded NestJS modules, the schema is versioned through Prisma migrations, and the infrastructure is reproducible through Terraform. Storefront theming is driven entirely by CSS variables on a single scoped wrapper, so a per-store re-skin requires no component-level changes.
Observability	Application logs are emitted to `stdout` and aggregated by Docker; the chat assistant offers a queryable view of the database for operational diagnostics.
Internationalisation	The interface is currently English-only; copy is centralised to enable future translation without code changes.

2.4 Use Case Diagrams

The use cases below were derived directly from the functional requirements in the previous section. They are presented from the most abstract (global) to the most granular (per actor), so that the reader can zoom from the platform-level overview into each role's specific interactions.

2.4.1 Global Use Case Diagram

Figure 2.1 groups the principal use cases of Cheebo around the three actors. Two dotted dependencies illustrate the cross-actor flows on which the rest of the chapter relies: a seller application *triggers* an administrative review (`«triggers»`), and product creation is automatically *verified by AI* (`«verified by AI»`).

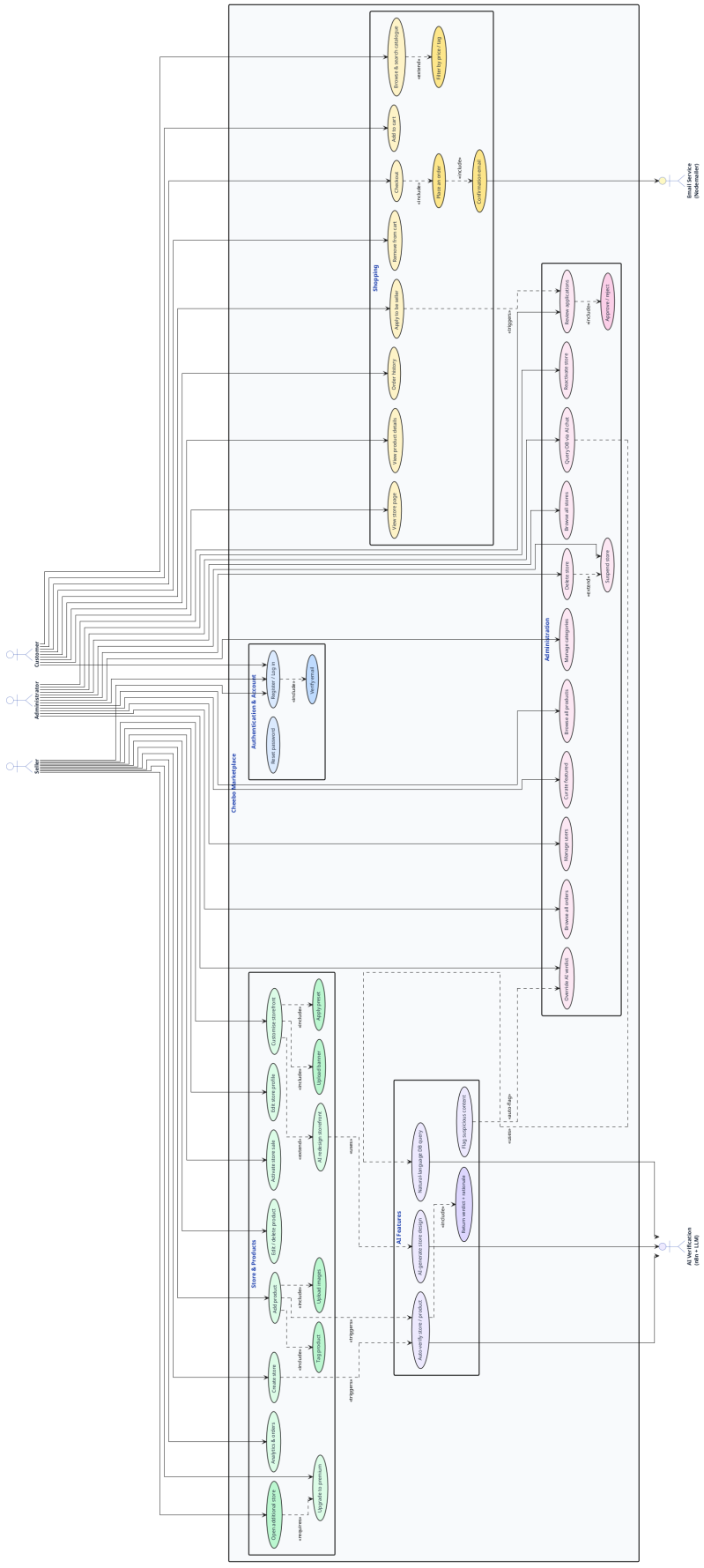

Figure 2.1: Global use case diagram (detailed)

2.4.2 Detailed Use Cases — Customer

The Customer view in Figure 2.2 expands the shopping path from registration to order placement. Registration «includes» email verification, and checkout «includes» both order placement and an authenticated session.

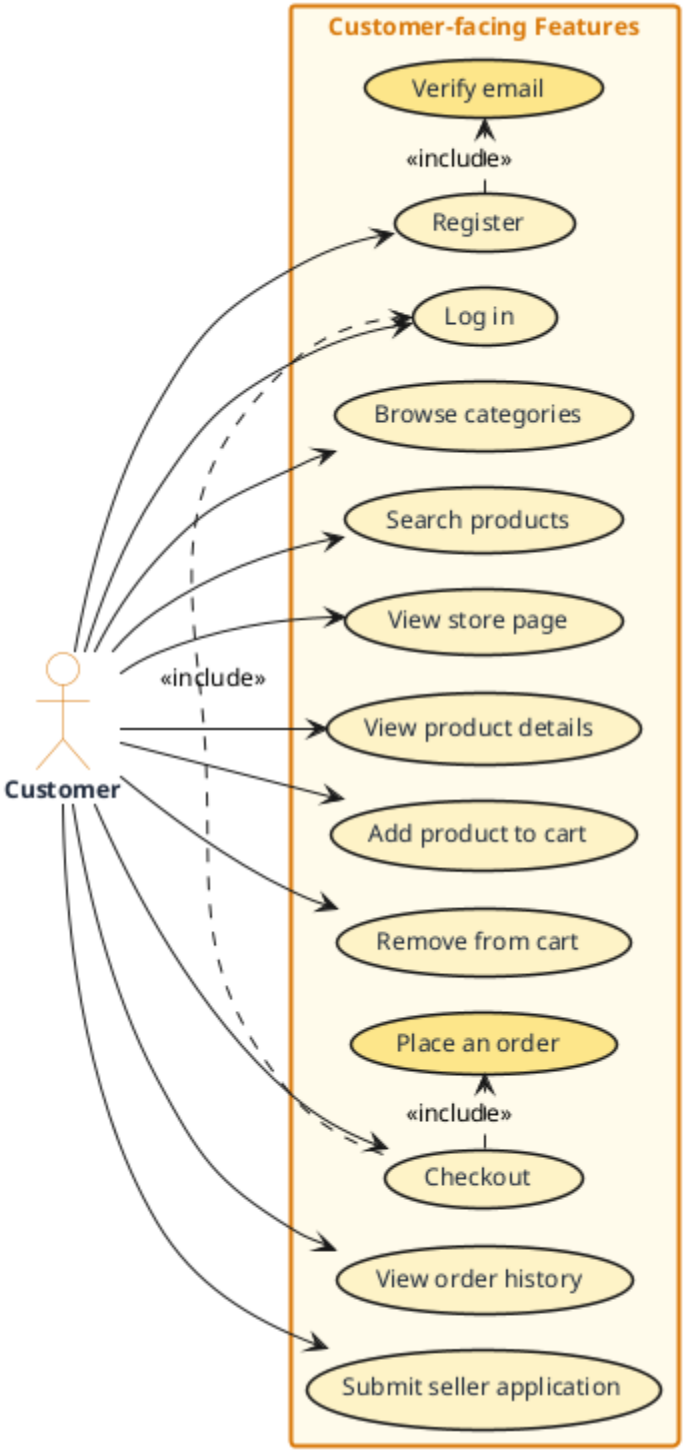

Figure 2.2: Customer use cases

2.4.3 Detailed Use Cases — Seller

Figure 2.3 shows that store creation depends on a prior approved seller application, and that opening additional stores depends on having activated the premium tier — the two gating conditions that govern the seller funnel.

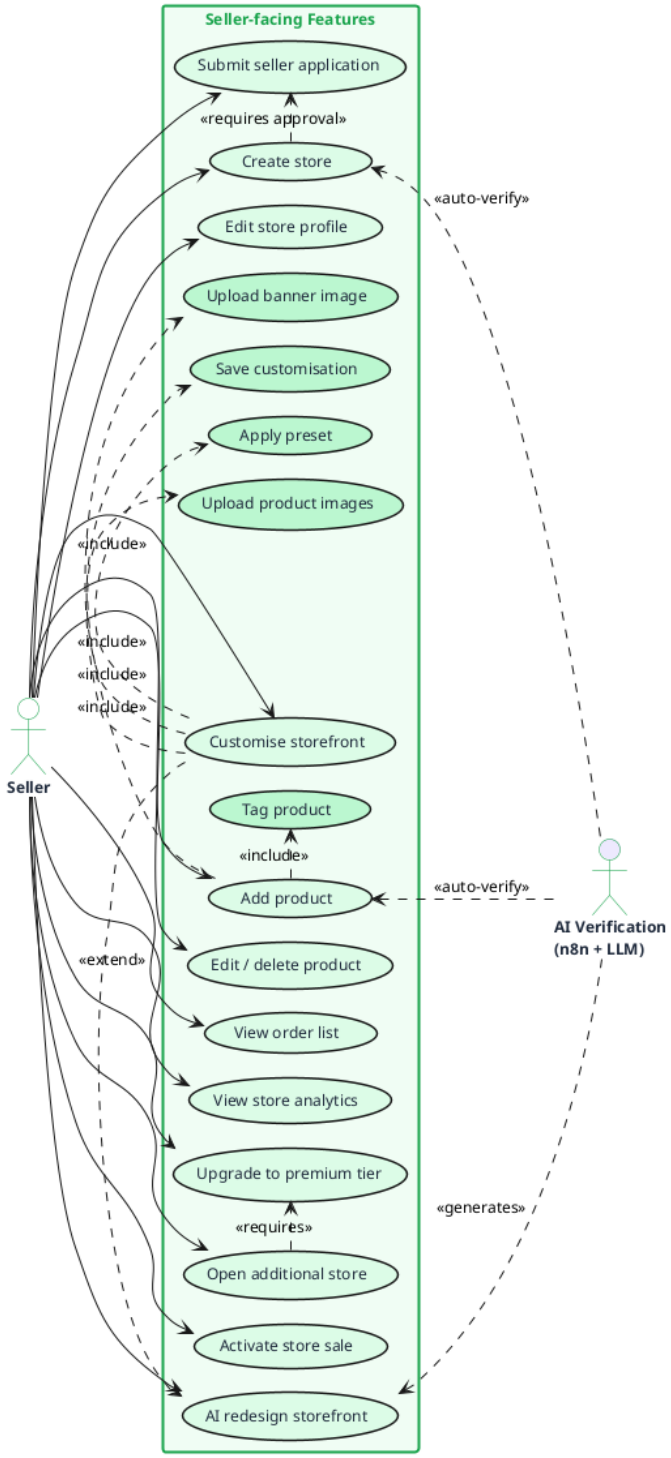

Figure 2.3: Seller use cases

2.4.4 Detailed Use Cases — Administrator

Figure 2.4 highlights the AI verification subsystem as a secondary actor that injects *auto-flag* events into the administration queue. The Administrator remains the final authority on every override, suspension and deletion.

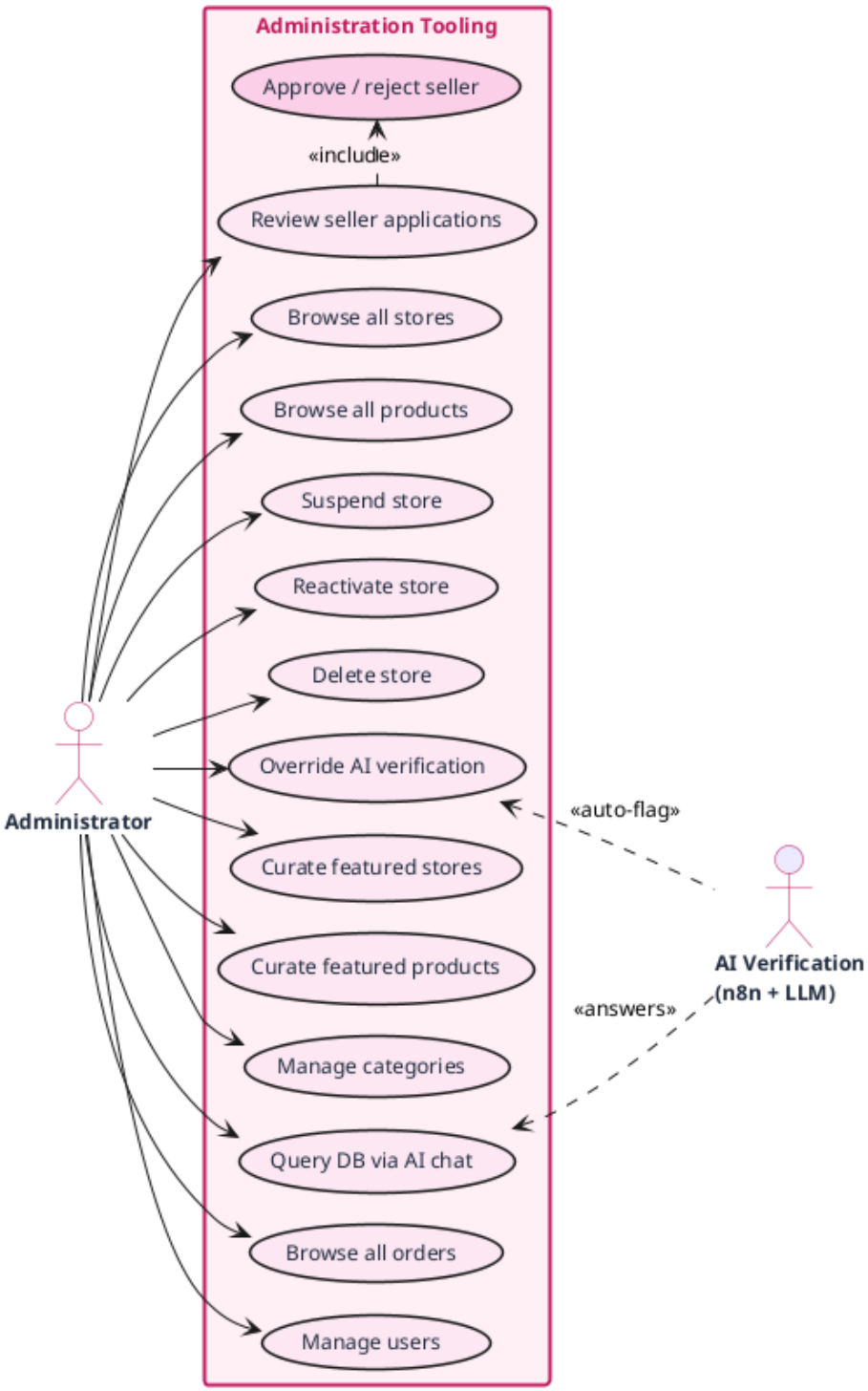

Figure 2.4: Administrator use cases

2.5 Sequence Diagrams

Three flows are particularly load-bearing for the architecture of the system: authentication, order placement, and AI-driven product verification. Each is described below with a UML sequence diagram generated from PlantUML sources stored under `diagrams/` in the project repository.

2.5.1 Registration, Email Verification and Login

Figure 2.5 traces the authentication flow. Registration creates a `User` row with `emailVerified=false` and triggers a Nodemailer message containing a one-time token; the user activates the account by clicking the link, which flips the flag and allows them to log in. Both Better-Auth and the NestJS layer participate, with sessions materialised as HTTP-only cookies returned by the API.

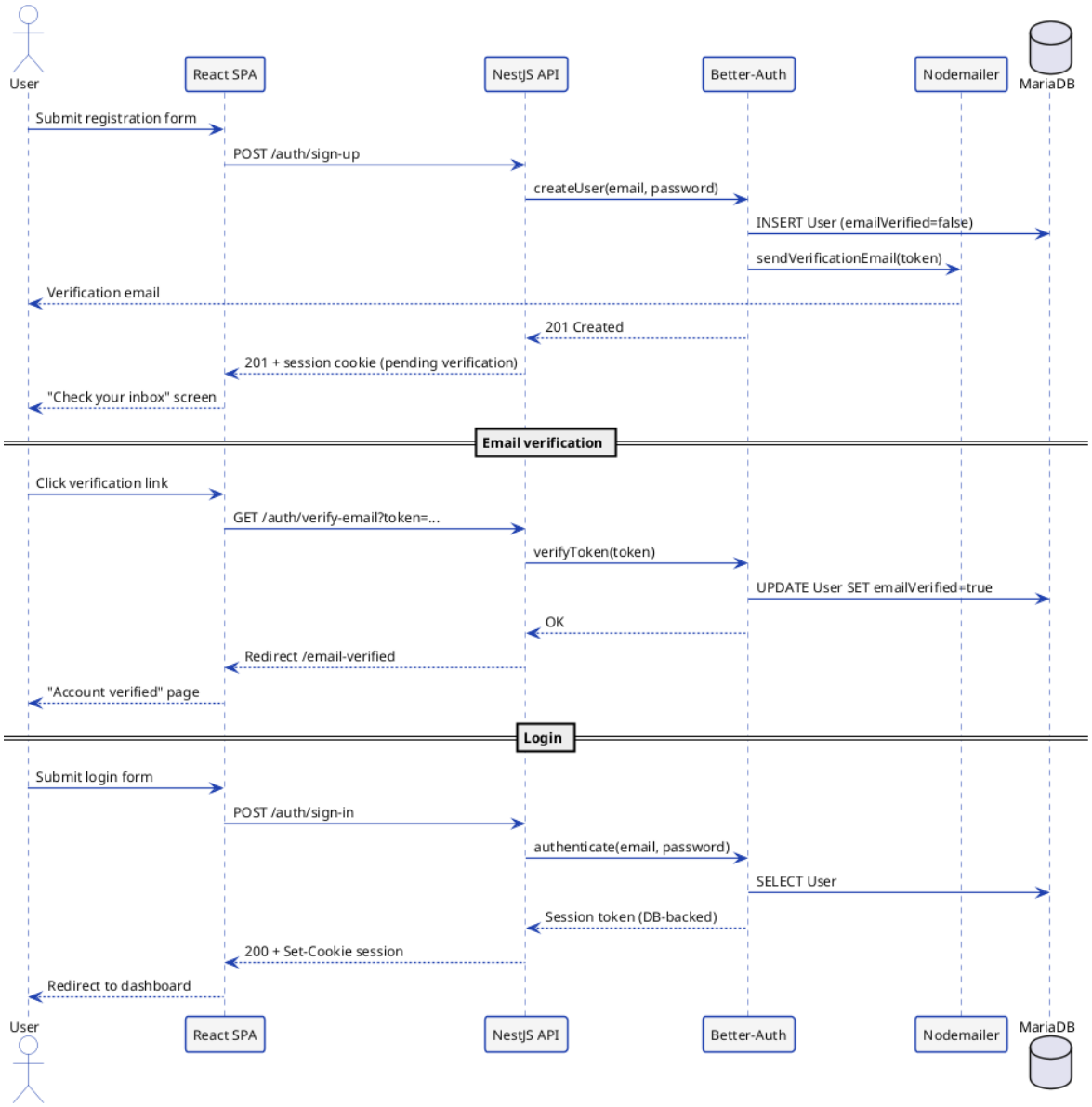

Figure 2.5: Sequence: registration, email verification and login

2.5.2 Order Placement

Figure 2.6 describes the checkout path. The cart is held client-side in a Zustand store and posted to the API as a single payload. The `AuthGuard` validates the session before the Order module persists the `Order` row and delegates the creation of line items to the OrderItems module; a confirmation email is sent before the response returns to the SPA, which then clears the cart store and redirects to the success page.

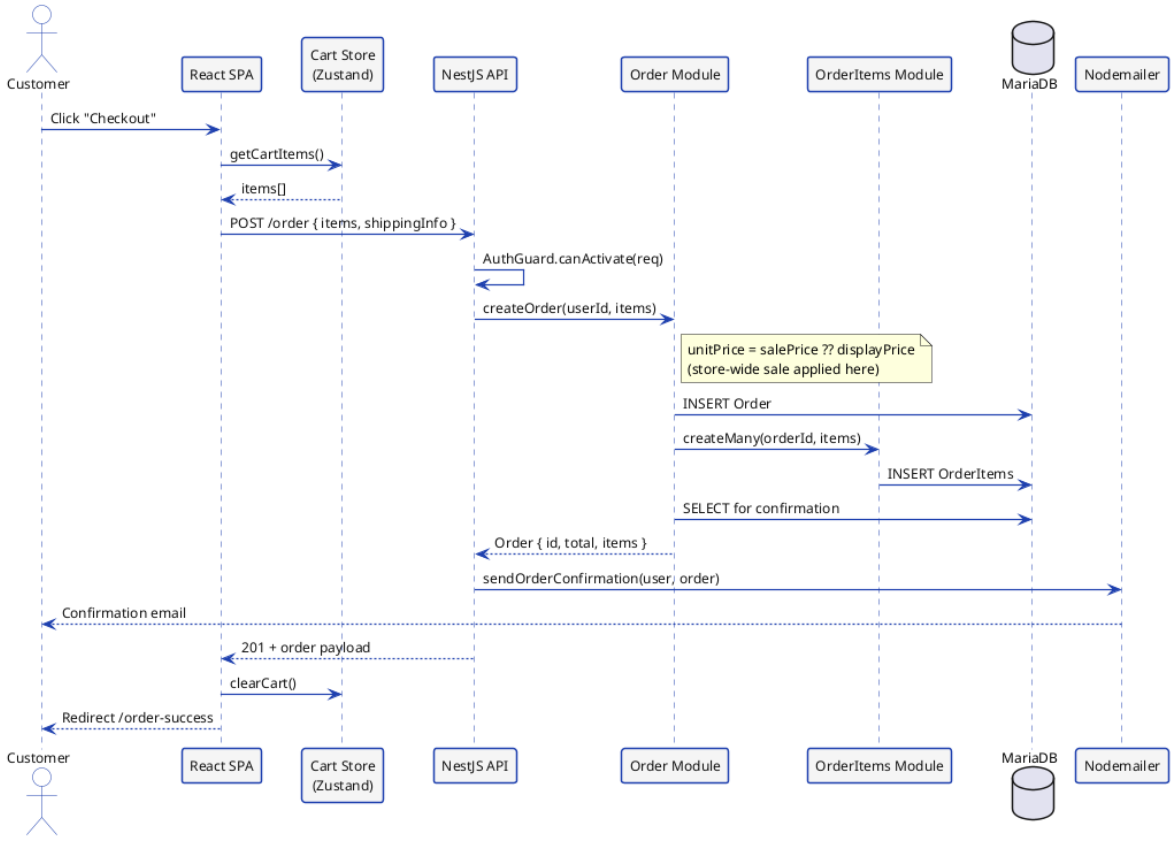

Figure 2.6: Sequence: order placement

2.5.3 Product Verification (AI + Administrator)

Figure 2.7 shows the dual-track verification process. Every product creation is forwarded to the `cheebo-ai-verify` webhook on n8n, which prompts a hosted LLM (Gemini or Groq) to apply the marketplace rules. The decision flows back to the backend through a callback endpoint and updates the `adminReviewed` column. The `alt` block at the bottom of the diagram captures the two outcomes: automatic approval (badge shown to the customer) and flagging (entry in the moderation queue, manual override by the Administrator).

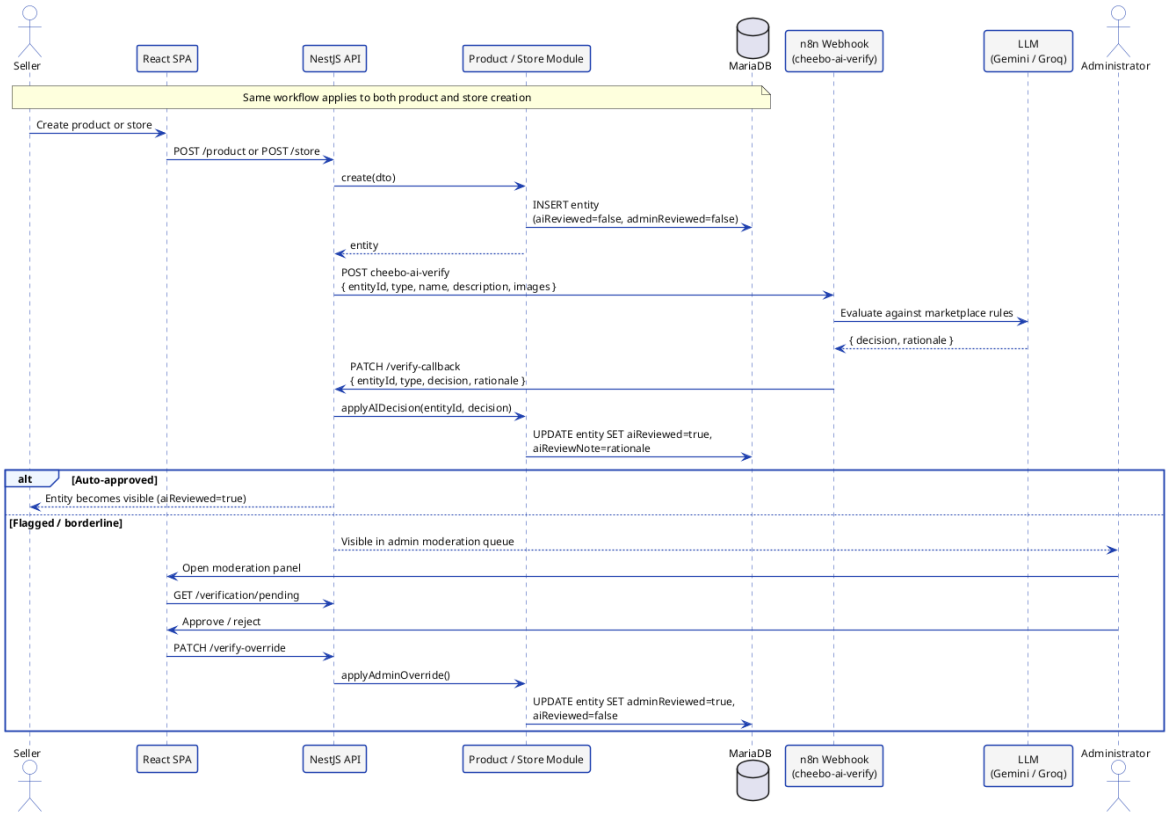

Figure 2.7: Sequence: product verification (AI and administrator)

2.6 Modelling Language

This project adopts the **Unified Modelling Language (UML)** to formalise both the static structure and the dynamic behaviour of the system. UML provides a standardised set of diagrams that represent software systems in a language-independent and tool-independent way. Three diagram types are used throughout this report:

- **Use Case diagrams** — describe the interactions between actors (Customer, Seller, Administrator) and the system, identifying the boundaries of the platform's functionality.
- **Sequence diagrams** — model the temporal exchanges of messages between system components for the three load-bearing flows: authentication, order placement, and AI-driven product verification.
- **Class diagrams** — represent the main domain entities, their attributes, and their associations, providing the structural blueprint from which the Prisma schema is derived.

All diagrams are authored in Mermaid (`.mmd` files) and rendered to PNG via the Mermaid CLI (`mmdc`), keeping diagrams version-controlled alongside source code.

2.7 Project Backlog

2.7.1 Scrum Team

- **Product Owner:** Mr. Mohammed Aziz Chibani (NeuralBey)
- **Scrum Master:** Mr. Mohammed Aziz Chibani (NeuralBey)
- **Development Team:** Nadim Jebali

2.7.2 MoSCoW Priority Legend

Each user story carries a **MoSCoW** priority and a **complexity** estimate, defined in Table 2.2.

Table 2.2: MoSCoW priority and complexity legend

MoSCoW Priority		
Must	*Must have*	Essential feature, required for the MVP.
Should	*Should have*	Important but not blocking.
Could	*Could have*	Desirable, included if time permits.
Won't	*Won't have*	Out of scope for this version.
Complexity		
1	Simple	Straightforward implementation, minimal effort.
2	Moderate	Average complexity, moderate effort.
3	Complex	Significant effort, elaborate logic.
5	Very complex	High effort, advanced technical challenge.

2.7.3 Product Backlog

Table 2.7.3 presents the product backlog structured as user stories following the template: *"As a <role>, I want <action> so that <benefit>"*. The columns **P** and **Cx** indicate respectively the MoSCoW priority and the estimated complexity.

Feature	ID	User Story	Priority	Cx
F1 Authentication	US01	As a visitor, I want to create an account with my email address so that I can access the platform.	**Must**	**2**
	US02	As a visitor, I want to receive a verification email so that I can activate my account securely.	**Must**	**2**

Continued on next page

Feature	ID	User Story	Priority	Cx
	US03	As a user, I want to sign in with my credentials so that I can access my personalised dashboard.	**Must**	**1**
	US04	As a user, I want to sign out so that I can protect my account on a shared device.	**Must**	**1**
F2 Store Management	US05	As a seller, I want to create a store with a name, description and logo so that customers can find and recognise my brand.	**Must**	**2**
	US06	As a seller, I want my store URL to use a human-readable slug so that it is easy to share.	**Must**	**2**
	US07	As a seller, I want to customise my storefront across seven sections (theme, announcement bar, banner, header, product grid, sidebar, footer) through a WYSIWYG visual editor so that I can see changes immediately and style my store to reflect my brand.	**Should**	**5**
	US08	As a seller, I want to apply a preset to my store so that I can get a professional look quickly.	**Should**	**2**
	US09	As a premium seller, I want to operate multiple stores so that I can segment my product offerings.	**Should**	**2**
	US34	As a seller, I want to activate a store-wide sale with a percentage discount so that all my products display discounted prices and a sale badge automatically across the storefront and search results.	**Should**	**2**
F3 Product Management	US10	As a seller, I want to create, edit and delete products so that I can keep my catalogue up to date.	**Must**	**2**
	US11	As a seller, I want to upload multiple product images so that customers can see my products clearly.	**Must**	**2**

Continued on next page

Feature	ID	User Story	Priority	Cx
	US12	As a seller, I want to assign tags to my products so that the recommendation engine can surface them.	**Should**	**1**
	US13	As a customer, I want to browse products by category so that I can discover relevant items.	**Must**	**1**
F4 Order System	US14	As a customer, I want to add products to a cart so that I can purchase multiple items at once.	**Must**	**2**
	US15	As a customer, I want to check out across multiple stores in a single order so that I do not need separate transactions.	**Must**	**3**
	US16	As a customer, I want to receive a confirmation email after placing an order so that I have a record of my purchase.	**Must**	**1**
	US17	As a seller, I want to view orders containing my products so that I can prepare and fulfil them.	**Must**	**2**
F5 Administration	US18	As an admin, I want to review seller applications so that I can approve legitimate merchants.	**Must**	**2**
	US19	As an admin, I want to suspend or delete any store so that I can enforce marketplace rules.	**Must**	**2**
	US20	As an admin, I want to curate featured stores and products so that the landing page promotes quality content.	**Should**	**1**
	US21	As an admin, I want to manage categories and sub-categories so that the catalogue stays organised.	**Must**	**1**
F6 AI Verification	US22	As the system, I want to automatically verify every new store and product via an LLM so that low-quality content is flagged without manual effort.	**Should**	**3**

Continued on next page

Feature	ID	User Story	Priority	Cx
	US23	As an admin, I want to override any AI verdict so that I remain the final authority on moderation decisions.	**Must**	**1**
	US24	As a customer, I want to see an AI-verified badge on trusted products so that I can shop with confidence.	**Won't**	**1**
	US35	As a seller, I want to regenerate my entire storefront design using a natural-language prompt sent to an AI agent so that I can obtain a professionally styled store without manual customisation.	**Could**	**3**
F7 Premium Tier	US25	As a seller, I want to subscribe to a premium plan so that I can unlock advanced storefront features.	**Should**	**3**
	US26	As a premium seller, I want access to all customisation controls so that I can fully personalise my store appearance.	**Should**	**3**
	US27	As the system, I want to preserve a seller's existing premium customisation after subscription expiry so that their store continues to look as designed.	**Could**	**2**
F8 CI/CD & Deployment	US28	As a developer, I want every service to run in a Docker container so that the environment is identical across development and production.	**Must**	**2**
	US29	As a developer, I want the backend and frontend images to be automatically built and pushed to Docker Hub on every push to main so that deployments are always up to date.	**Must**	**2**
	US30	As a developer, I want the production droplet to be provisioned automatically via Terraform so that the infrastructure is reproducible and version-controlled.	**Should**	**3**

Continued on next page

Feature	ID	User Story	Priority	Cx
	US31	As a developer, I want the application to be deployed to DigitalOcean via SSH on every push to main so that updates reach production without manual intervention.	**Must**	**3**
	US32	As a developer, I want all secrets and environment variables to be managed through GitHub Actions secrets so that no credentials are stored in the repository.	**Must**	**1**
	US33	As a developer, I want the domain DNS A records to be updated automatically to the new droplet IP via the DigitalOcean Terraform provider so that the application is always reachable at `nadimjebali.engineer`.	**Should**	**2**

Key Takeaways

- Three actors — Customer, Seller, Administrator — are modelled as roles on a single `User` table; an account can be promoted across roles without re-registering.
- Functional requirements are grouped into seven domains (authentication, store, product, order, admin dashboard, AI verification, featured curation), each mapped one-to-one with a backend NestJS module.
- The non-functional contract privileges *security*, *performance* and *maintainability* as first-order qualities, with concrete targets stated in Table 2.1.
- The Product Backlog comprises 35 user stories across 8 features, prioritised with MoSCoW and estimated for complexity.

Chapter Conclusion

This chapter formalised the requirements of the Cheebo platform. The three system actors were identified and their responsibilities delimited. Functional requirements were grouped into seven domains — authentication, store management, product management, orders, administration, AI verification, and featured content curation — and expressed both as structured requirement statements and as a user-story Product Backlog with MoSCoW priorities and complexity estimates. Non-functional constraints were captured in a consolidated table. UML use-case and sequence diagrams illustrated the system boundaries and the three load-bearing flows. The following chapter builds on these specifications to present the system design and architecture.

Chapter 3 Design and Architecture

This chapter translates the requirements identified in Chapter 2 into a concrete system design. It opens with a global three-tier view, then drills down into the NestJS module layout, the React SPA structure, the Docker Compose topology, the database schema, the REST API contract, the three n8n workflows that carry the platform's AI features, and finally the UML class model that unifies all the design decisions.

3.1 Global Architecture

3.1.1 3-Tier Architecture Overview

Cheebo follows a classic **three-tier architecture** that separates the user interface (presentation), the business logic (application), and the persistent data (data) into three independent layers. This separation is the structural foundation on which every other design decision in this chapter builds: it lets each tier evolve at its own pace, scales the backend horizontally without touching the frontend, and keeps the database schema decoupled from the public REST contract.

Figure 3.1 shows the three tiers and the auxiliary components that orbit them. End users reach the platform exclusively through HTTPS to an **Nginx** reverse proxy, which serves the static React bundle and forwards every `/api/*` request to the NestJS API container over the Docker bridge network. The NestJS application implements the entire business logic — authentication, catalogue management, order processing, moderation — and persists state to a single **MariaDB 11** instance through the **Prisma** ORM. File uploads are handled by **Multer** and stored on a Docker volume; transactional emails are dispatched through **Nodemailer** over SMTP.

Sitting alongside the backend rather than inside it, **n8n** hosts the three AI workflows (verification, database chat, AI redesign). The backend reaches n8n via internal HTTP webhooks; n8n in turn calls the LLM providers (Groq and Gemini) over the public internet. This separation isolates AI orchestration — prompt templates, model selection, retry logic — from the core business logic, so it can be modified through the n8n editor without redeploying the API.

Figure 3.1 summarises the topology. End users on the left reach Nginx, which serves the SPA and forwards API traffic to the NestJS backend; the backend in turn talks to MariaDB, Nodemailer and Multer, and exchanges webhooks with n8n on the right,

which itself calls the LLM providers.

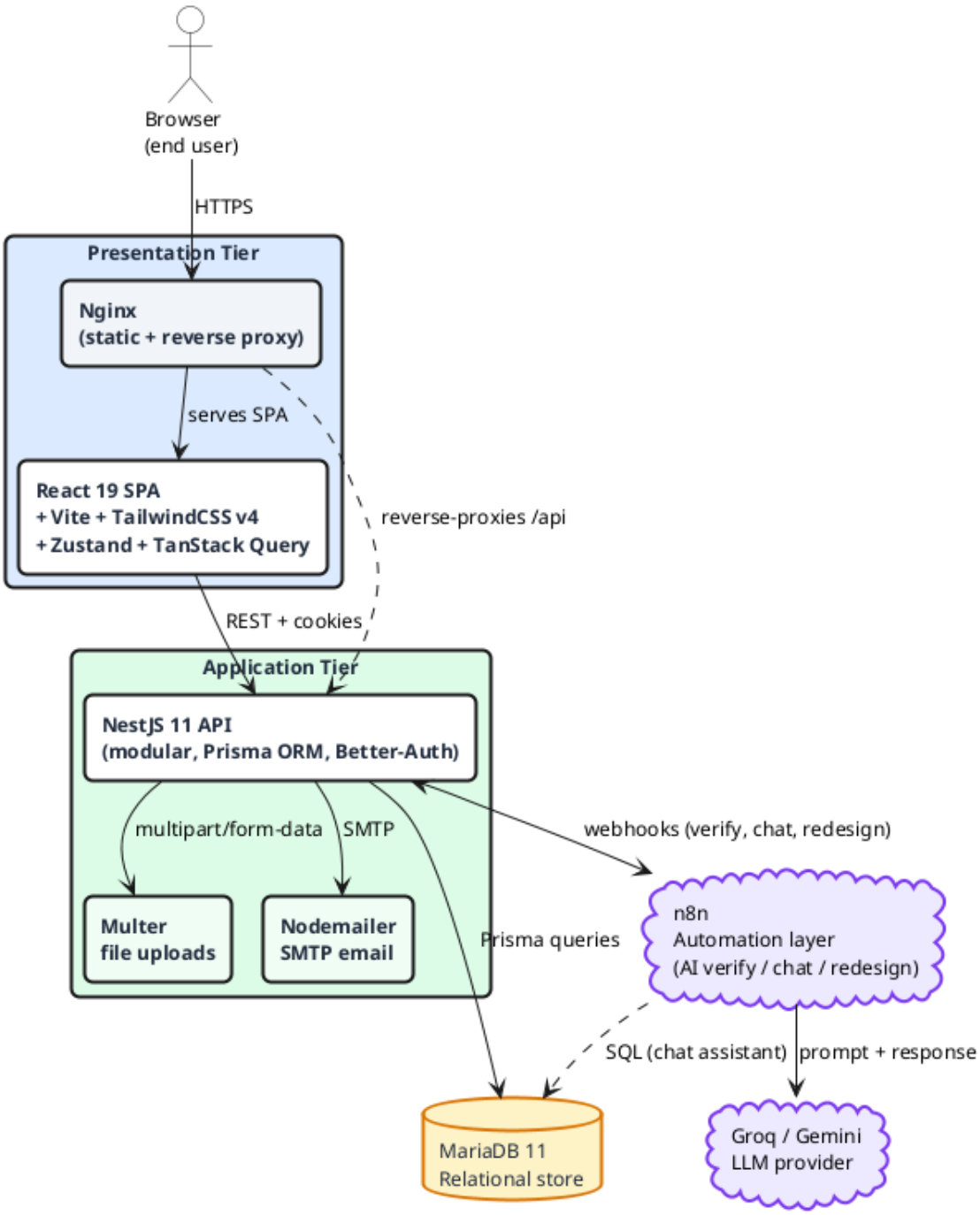

Figure 3.1: Global system architecture (three tiers + n8n automation layer)

3.1.2 N8N as an Automation Layer

n8n was introduced as a deliberate architectural choice rather than an ad-hoc tool. Three properties made it the right fit for Cheebo's AI features:

1. **Workflow-as-data.** Every AI feature is expressed as a JSON workflow that can be exported, version-controlled in the repository (`n8n-workflows/`), and re-imported

on any n8n instance. The workflow becomes a first-class artefact of the system, reviewed alongside the application code.

2. **Provider abstraction.** The NestJS backend never imports an LLM SDK. Switching from Gemini to Groq, or adding a new provider for a specific node, is a configuration change inside the n8n editor with no NestJS recompilation. This decoupling also keeps API keys out of the backend container.
3. **Native webhook orientation.** n8n exposes any workflow as a webhook endpoint (`/webhook/<name>`) and supports synchronous *Respond to Webhook* for request/response flows and asynchronous patterns for fire-and-forget jobs. This matches exactly the three integration patterns Cheebo needs: asynchronous (verification fires on entity creation and writes back later), synchronous (AI redesign returns the `StoreCustomization` to the seller's editor in the same call), and conversational (the database chat assistant streams messages between the admin and the LLM).

The trade-off accepted in exchange for these benefits is one additional container in the production stack and one additional persistence volume (`n8n_data`) to back up. Both are justified by the maintainability gains.

3.2 Detailed Technical Architecture

3.2.1 NestJS Modular Architecture

The backend is structured as a set of **feature modules**, each encapsulating its own controller, service, DTOs, and Prisma queries. Every module declares its own `@Module()` decorator and exports the providers consumed by sibling modules; this strict separation is what allows the dependency graph in Figure 3.2 to remain acyclic in a system that hosts almost twenty modules.

Cross-cutting modules. Four modules are imported by virtually every feature: `PrismaModule` exposes the singleton Prisma client; `AuthModule` provides the `AuthGuard` and the `@Roles()` decorator that gate every protected route; `NotificationModule` centralises in-app notifications dispatched through the EventEmitter; `SettingsModule` stores platform-wide configuration that administrators can update without redeploying.

Feature modules grouped by domain.

- **User and application:** `UserModule` and `SellerApplicationModule` handle profile management and the seller onboarding workflow.
- **Storefront:** `StoreModule`, `StoreStatusLogModule`, `ProductModule`, `ProductImageModule`, and `ProductTagModule` cover everything a seller owns.

- **Catalogue:** `CategoryModule` and `SubCategoryModule` maintain the taxonomy that the product grid filters by.
- **Commerce:** `OrderModule` and `OrderItemsModule` create orders and persist line items with the discounted unit price computed at checkout.
- **Discovery and analytics:** `HomeModule` powers the landing page (featured stores, featured products, tag-based recommendations); `AnalyticsModule` computes per-store revenue and visitor metrics.
- **AI integration:** `ChatModule` relays administrator messages to the database chat workflow; `N8nModule` centralises the webhook calls used by both the verification pipeline and the AI redesign feature.

This layout maps one-to-one onto the functional requirement domains of Section 2.2, which makes it easy to trace a requirement to a single module and vice versa.

Figure 3.2 visualises the resulting module catalogue. The cross-cutting providers are shown in blue at the top, the domain-specific feature modules are grouped into five coloured packages, and the AI integration package on the right collects the modules that exchange webhooks with n8n.

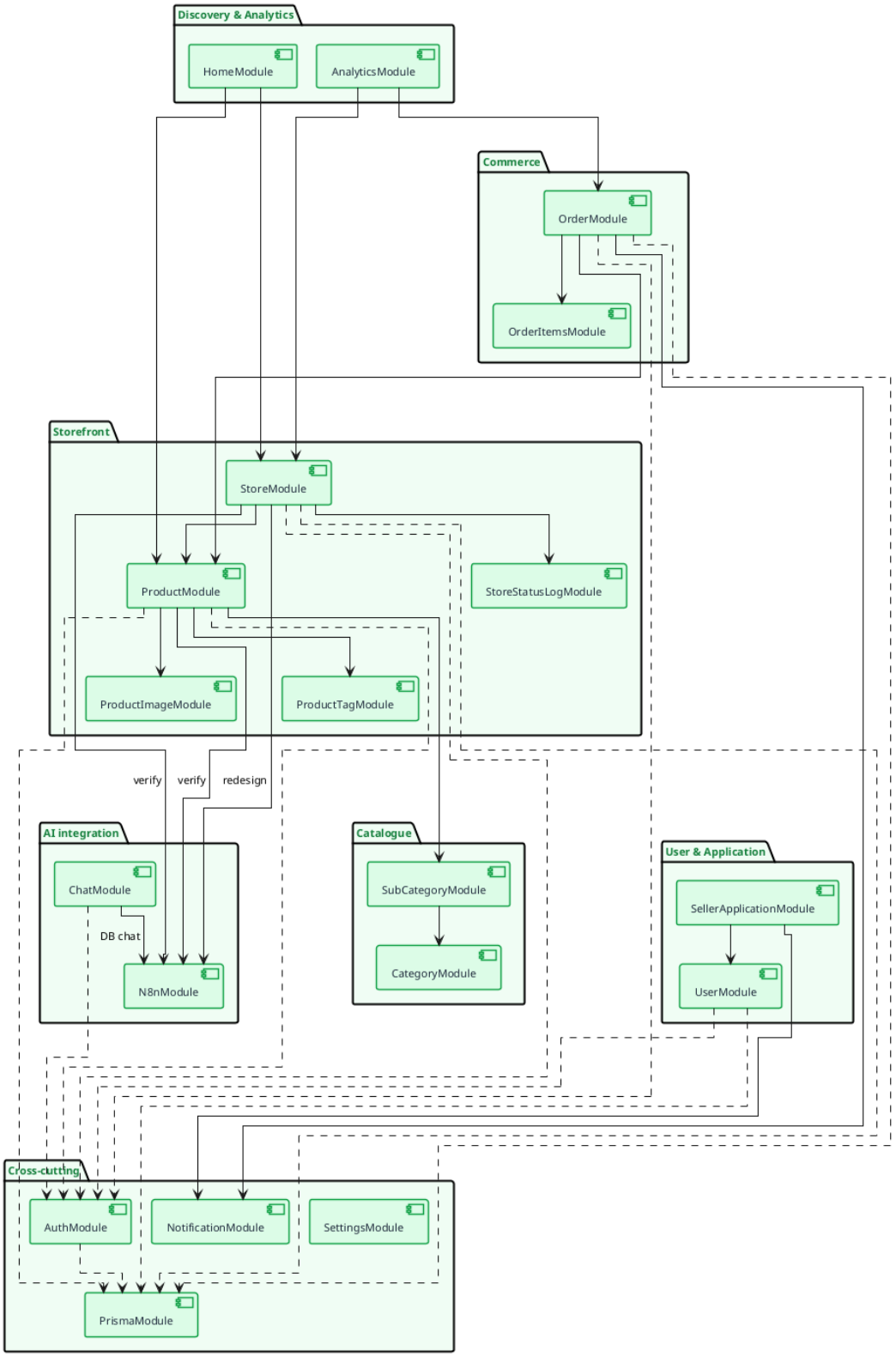

Figure 3.2: NestJS modular architecture: groups of feature modules and their dependencies

3.2.2 React SPA Architecture

The frontend is a **Single-Page Application** built with React 19 and bundled by **Vite**. The codebase under `frontend/src/` is split into seven top-level folders, each playing a clearly defined role:

- `pages/` — one file per route, including the public catalogue (`Home`, `ProductsPage`, `StorePage`, `ProductPage`), the checkout flow (`CartPage`, `CheckoutPage`, `OrderSuccessPage`), authentication (`Login`, `Register`, `EmailVerified`), and a nested `dashboard/` subtree split by role (seller, admin).
- `components/` — reusable building blocks (`ProductCard`, `StoreCard`, `SearchBar`, `AiChatFab`, and the `ui/` folder of `shadcn/ui` primitives).
- `api/` — one TypeScript file per backend module (`store.api.ts`, `product.api.ts`, ...) that wraps the typed Axios client and exposes a single object of named functions, mirroring the NestJS layout.
- `stores/` — Zustand stores for client-side state (`auth.store`, `cart.store`, `admin.store`, `category.store`, `customization.editor.store`).
- `hooks/` — TanStack React Query hooks that wrap the API layer, providing caching, deduplication, and background refetching.
- `lib/` — framework-agnostic utilities (Axios instance, customisation hydration, error formatter).
- `types/` and `constants/` — domain types shared with the backend DTOs and constant tables (roles, statuses, MIME lists).

State management follows a strict two-tier convention: **server state** (anything originating from the API) lives in **TanStack Query** caches keyed by the API path; **client-only state** (cart contents, theme preference, draft store customisation in the WYSIWYG editor) lives in **Zustand** stores. This division eliminates the classic React anti-pattern of duplicating server data into a Redux store and then fighting cache invalidation.

The storefront editor itself is a separate sub-architecture built on **Craft.js**: the canvas, the section palette, and the inspector panel are wired around a single `<Editor>` provider, and a Zustand store mirrors the Craft editor state so that the *Save* button can submit a stable JSON payload to the backend.

3.2.3 Docker Compose Services Diagram

The full application stack is defined in a single `docker-compose.yml` file at the root of the monorepo. Four services are declared (Figure 3.3):

- **db** — official `mariadb:11` image with a named volume `db_data` and a health-check that gates the backend's start-up.
- **backend** — the pre-built `sirvinny/cheebo-backend:latest` image, configured from environment variables (database credentials, Better-Auth secret, SMTP settings, n8n webhook URLs, premium-tier limits) and mounting an `uploads` volume for stored images.
- **frontend** — the pre-built `sirvinny/cheebo-frontend:latest` image (Nginx serving the React bundle) with the only published port, `80:80`.
- **n8n** — the official `n8nio/n8n:latest` image with a named volume `n8n_data` preserving workflows and credentials across restarts.

All services share a single Docker bridge network that is created implicitly by Compose; only the frontend port is published. The backend, database, and n8n are never reachable directly from the public internet — the backend's URL is `http://backend:3000` from inside the bridge network, and the database is reachable as `db:3306`. This single-host topology is intentional: the production droplet specifications (Section 5.4.1) size for it, and the architecture is straightforward to migrate to a managed database or a Kubernetes deployment later.

Figure 3.3 shows the resulting topology on the production host: the four containers sit on a shared Docker bridge network, only the frontend publishes a port to the public internet, and each stateful service writes to its own named volume.

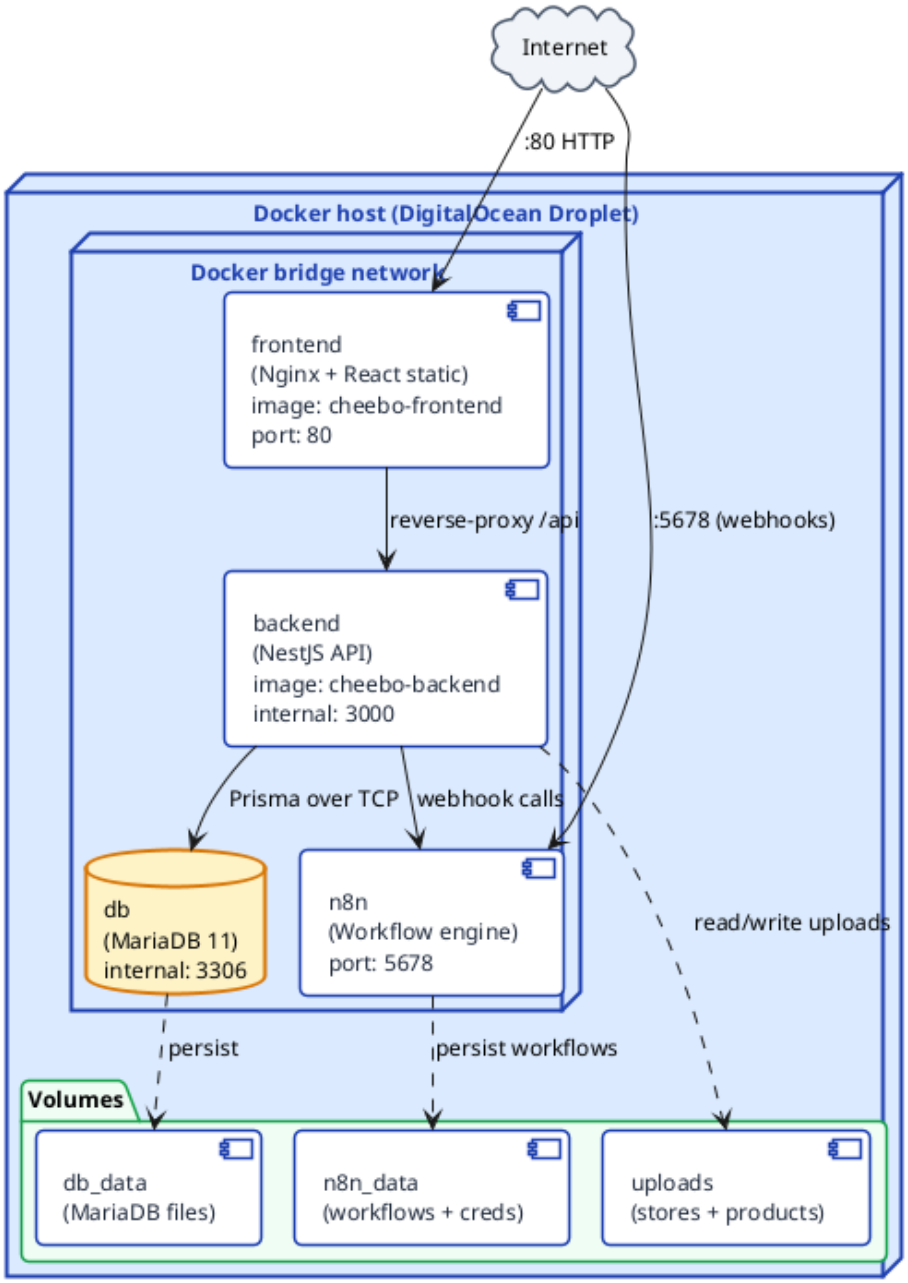

Figure 3.3: Docker Compose services on the production host

3.3 Database Design

3.3.1 Entity-Relationship Diagram

Figure 3.4 presents the entity-relationship diagram of the Cheebo database. The schema is organised around the `User` entity, which acts as the entry point for three independent branches: authentication (`Session`, `Account`, `Verification`), commerce (`Order` and through it `OrderItems`), and storefront ownership (`SellerApplication` and `Store`). Each `Store` in turn owns its own subtree of `Product`, `ProductImage`, `ProductTag` and `StoreStatusLog` records, while the `Category`/`Subcategory` hierar-

chy lives orthogonally and is referenced by every product.

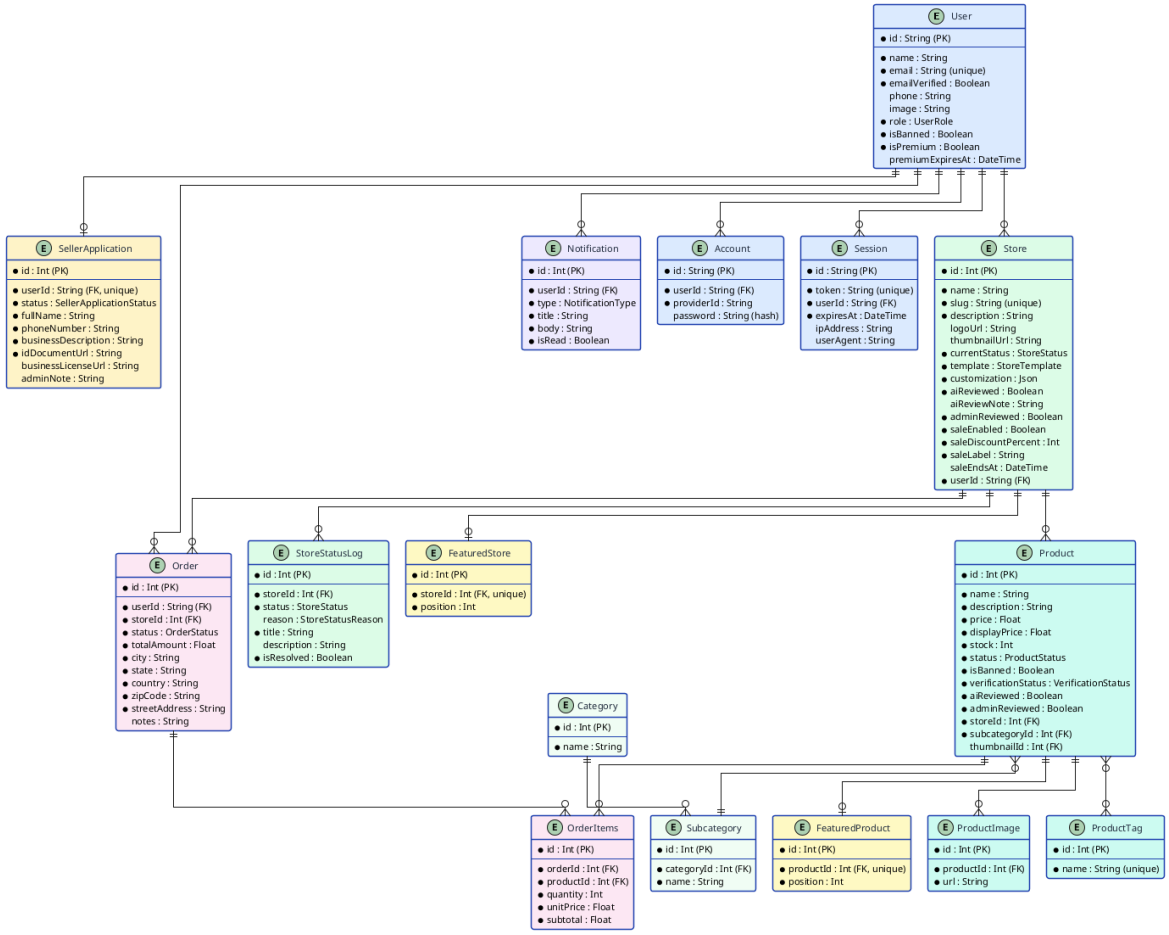

Figure 3.4: Entity-Relationship diagram of the Cheebo database

3.3.2 Prisma Schema Overview

The Prisma schema is split into eighteen files under `backend/prisma/schema/`, one per logical entity, plus an `enums/` folder gathering every enumeration in a single place. This split takes advantage of the `multi-file schema` preview feature introduced in Prisma 5 and keeps each file under one screen, which makes diff review on pull requests far easier than a single multi-thousand-line schema.

- **Authentication** (`user.prisma`, `auth.prisma`) — four models: `User`, `Session`, `Account`, `Verification`. The non-`User` tables are managed entirely by Better-Auth and are exposed to Prisma read-only.
- **Storefront** (`store.prisma`, `store-status-log.prisma`, `store-template.prisma`) — the `Store` model carries a JSON `customization` column that holds the Craft.js editor state, plus the four sale fields (`saleEnabled`, `saleDiscountPercent`, `saleLabel`, `saleEndsAt`).

- **Product catalogue** (`product.prisma`, `product-image.prisma`, `product-tag.prisma`, `category.prisma`, `subcategory.prisma`) — five models capturing the catalogue, with two-level category taxonomy and many-to-many tags via an implicit join table.
- **Commerce** (`order.prisma`, `order-items.prisma`) — two models with the unit price persisted on each line so that subsequent price changes never alter historical orders.
- **Curation** (`featured.prisma`) — two models (`FeaturedStore`, `FeaturedProduct`) with a `position` integer driving the landing-page ordering.
- **Workflow** (`seller-application.prisma`, `notification.prisma`, `setting.prisma`) — application reviews, in-app notifications, and platform-wide settings respectively.

The complete schema, including indexes and cascade rules, is reproduced in Appendix A.

3.3.3 Explanation of Key Relationships

A handful of relationships carry most of the load and deserve a dedicated note.

User → Store → Product (1:N:N). A `User` with the `SELLER` role can own one store on the free tier and several on the premium tier; each store owns an unbounded number of products. The chain is materialised by two foreign keys (`Store.userId`, `Product.storeId`) both indexed. The cascading-delete rule on `Product` ensures that suspending or deleting a store cleanly removes the products it contained.

User ↔ SellerApplication (1:0..1). A user has at most one open seller application at any time. The `userId` column is `@unique`, which prevents duplicate submissions at the database level rather than at the service layer. The `Cascade` delete rule on the FK ensures that deleting a user automatically purges the application row.

Order → Store and Order → OrderItems. A single multi-store checkout creates **one `Order` per store** (so the seller has a clean per-store view) but a single shopping flow for the customer. Each `OrderItems` row carries the unit price that was visible at checkout time — this is the discounted price when the store had an active sale, computed by the `Product` service before the order is persisted. Subsequent price changes never rewrite history.

Product ↔ ProductTag (M:N). Prisma's *implicit* many-to-many relation is used here: Prisma manages an internal join table `_ProductTags` that does not appear in the schema file, which keeps the model lean while still allowing efficient queries by tag for the recommendation engine.

Verification flags on Store and Product. Both entities carry two independent boolean flags: `aiReviewed` (set automatically by the n8n verification workflow) and `adminReviewed` (set when an administrator manually overrides the AI verdict). This dual-flag design preserves the audit trail of who — machine or human — approved a piece of content, and lets the moderation queue filter on `aiReviewed = false AND adminReviewed = false` to find genuinely pending items.

Sale fields on Store. The four sale fields live on `Store` rather than on a separate `Sale` entity. This denormalisation is deliberate: sales are always store-wide, only one is active at a time, and the fields are read on every product query. A separate join would have added a query without buying any expressive power.

3.4 API Design

3.4.1 REST API Design Principles

The Cheebo backend exposes a single **REST API** over JSON that the React SPA consumes through a typed Axios client. Six principles guided every route declaration:

1. **Resource-oriented URLs.** Routes name nouns (`/store`, `/product`, `/order`) and use sub-paths for actions on a specific resource (`/store/:id/sale`, `/store/:id/ai-redesign`). Verbs in URLs are avoided.
2. **HTTP verb semantics.** `GET` for safe reads, `POST` for creation, `PATCH` for partial updates, `PUT` for full replacements, `DELETE` for removal. Status codes follow the same convention — `201 Created` on `POST`, `204 No Content` on `DELETE`, `400` for validation errors, `401` for unauthenticated access, `403` for role violations.
3. **Validated DTOs.** Every body- or query-bearing endpoint declares a `class-validator` DTO with `forbidNonWhitelisted: true`, so any extra field in the payload is rejected at the framework layer before the controller handler executes.
4. **Role-based access control.** Protected routes are decorated with `@UseGuards(AuthGuard, RolesGuard)` and the `@Roles(...)` decorator listing the allowed roles. Anonymous endpoints (catalogue browsing, store page) carry `@Public()`.
5. **Cursor- and page-based pagination.** List endpoints accept `page` and `limit` query parameters and always return a uniform `PaginatedResult<T>` envelope (`{ data, total, page, limit }`).
6. **Stable error contract.** Failures are serialised as `{ statusCode, message, error }` so that the frontend's `getApiError` helper can handle both single-string and array-of-strings messages from validation pipes without branching on the endpoint.

OpenAPI documentation is generated automatically by the `@nestjs/swagger` module and served at `/api/docs` when the `SWAGGER_PATH` environment variable is set, which is useful in development and during defence preparation but disabled in production.

3.4.2 Endpoint Structure

Table 3.1 lists representative endpoints grouped by module. The complete list of around one hundred routes is reproduced in Appendix B.

Table 3.1: Main REST API endpoints, grouped by module

Method	**Route**	**Description**
Authentication (Better-Auth, mounted under /api/auth)		
POST	/auth/sign-up	Register a new user, send verification email
POST	/auth/sign-in	Authenticate and issue session cookie
POST	/auth/sign-out	Invalidate the current session
GET	/auth/verify-email	Confirm an email verification token
Seller application		
POST	/seller-application	Submit a new application
GET	/seller-application/my	Read my own application
GET	/seller-application	(admin) List every application
PATCH	/seller-application/:id	(admin) Approve, reject, or request info
Store		
GET	/store	List public stores (paginated, searchable)
GET	/store/by-slug/:slug	Resolve a store by its public slug
POST	/store	Create a store (triggers AI verification)
PATCH	/store/:id/customization	Save Craft.js storefront configuration
PATCH	/store/:id/sale	Activate, edit or disable the store-wide sale
POST	/store/:id/ai-redesign	Request an AI-generated full redesign
PATCH	/store/:id/status	(admin) Suspend, reactivate or delete
Product		
GET	/product	List products (filters, sort, pagination)
GET	/product/by-store/:id/cards	Lightweight product cards for storefront
GET	/product/recommended	Tag-based recommendations
GET	/product/search	Full-text search across catalogue
POST	/product	Create a product (triggers AI verification)
PATCH	/product/:id/verify	(admin) Override AI verdict
Order		
POST	/order	Place an order (multi-store checkout)
GET	/order/my	Read the current user's order history
GET	/order	(seller) Orders containing my products
GET	/order/dashboard	(admin) Browse all orders
Featured content		

3.4.3 Authentication Flow (Better-Auth)

Authentication is handled entirely by the **Better-Auth** library, integrated into NestJS through a thin adapter module. Better-Auth was preferred over rolling a custom Passport strategy because it ships with email/password, email verification, session persistence in the database, and cookie-based session transport out of the box — everything Cheebo needs without writing any cryptography. Sessions are opaque DB-backed tokens (not JWTs), which makes server-side revocation a one-line update on the `Session` table.

Figure 3.5 details the three-stage flow: registration (creates the `User` row, sends the verification email, returns a session cookie marked *pending verification*), email verification (flips `emailVerified=true`), and login (validates credentials and issues a fresh session). Every subsequent protected request carries the `better-auth.session_token` cookie, which the NestJS `AuthGuard` validates against the `Session` table on each call.

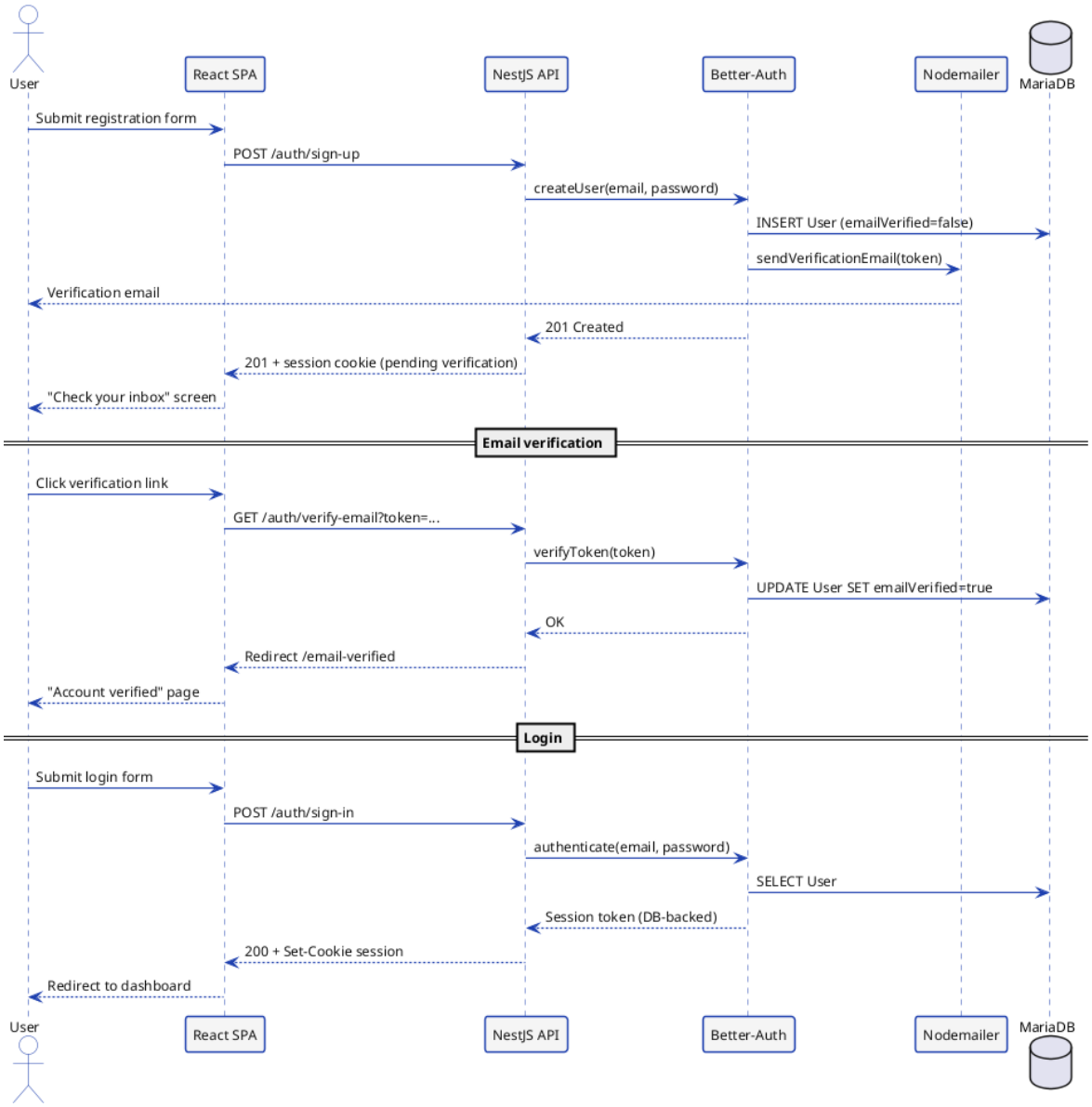

Figure 3.5: Better-Auth authentication flow (registration, verification, login)

3.5 N8N Workflow Design

Cheebo's AI features are carried by three independent n8n workflows deployed on the same engine. Each one is exposed as an HTTP webhook that the NestJS backend invokes; their JSON definitions live in `n8n-workflows/` in the repository and are reproduced in Appendix C. The three workflows differ in their trigger, integration pattern, and downstream model:

- **AI verification** — fired automatically when a seller creates a store or a product, asynchronous response through a callback endpoint.
- **Database chat assistant** — triggered by an administrator's chat message, conversational request/response.

- **AI storefront redesign** — triggered explicitly by a seller from the Craft.js editor, synchronous response returning a `StoreCustomization` JSON object.

3.5.1 AI Verification Workflow

Figure 3.6 shows the n8n workflow responsible for automatic store and product verification. Upon creation of a new entity, the backend fires a webhook to n8n, which calls the hosted LLM (Gemini or Groq), evaluates the content against the marketplace rules, and posts the verdict back to the `/callback` endpoint.

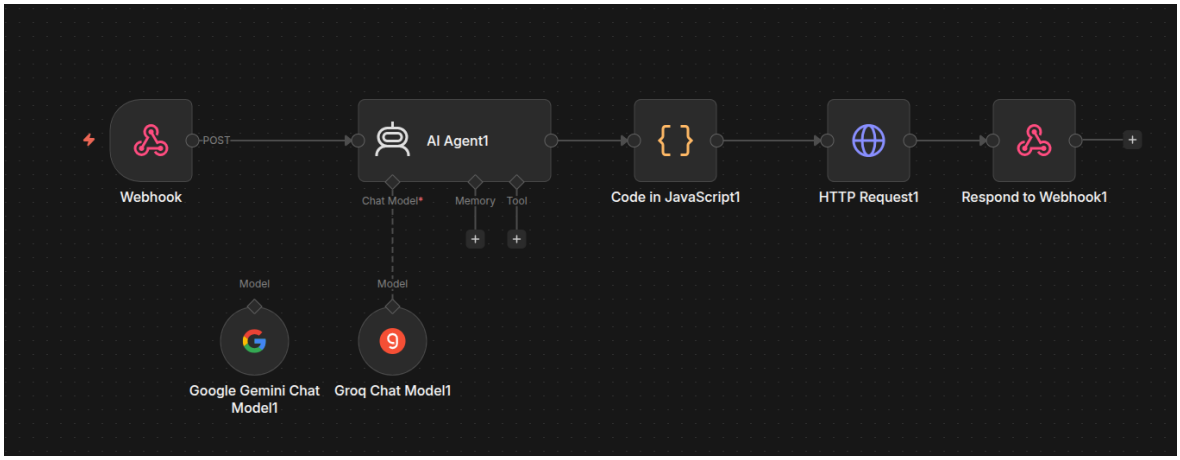

Figure 3.6: N8N workflow — AI verification

3.5.2 Database Chat Assistant Workflow

Figure 3.7 illustrates the database chat assistant workflow. Administrator messages are forwarded to n8n, which uses an LLM to translate the natural-language query into SQL, executes it against the MariaDB instance, and returns a formatted response.

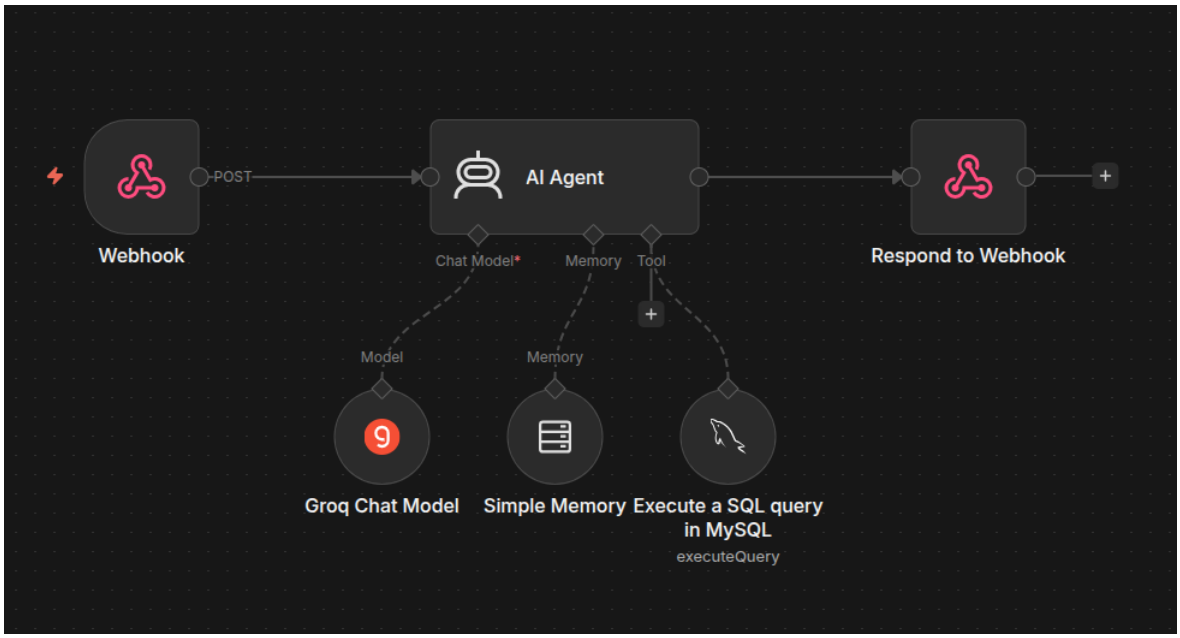

Figure 3.7: N8N workflow — Database chat assistant

3.5.3 AI Storefront Redesign Workflow

The third n8n workflow handles AI-assisted storefront redesign. Unlike the verification workflow, which is triggered automatically on entity creation, this workflow is initiated explicitly by the seller from the Craft.js editor. The seller provides a short prompt describing the desired aesthetic; the backend forwards it to the **cheebo-store-redesign** webhook along with the store name and description as context.

The workflow is composed of three nodes: a **Build Prompt** Code node that assembles the system instructions and enforces the `StoreCustomization` JSON schema; an **AI Agent** node connected to the Groq `llama-3.3-70b-versatile` model that produces the design; and a **Parse & Sanitise** Code node that validates hex colour values, clamps numeric fields, and applies safe defaults for any missing properties. The sanitised object is returned synchronously to the backend, which persists it and streams it back to the editor.

Figure 3.8 shows the workflow as it appears in the n8n editor, with the webhook trigger on the left, the Build Prompt Code node, the AI Agent connected to the Groq chat model, the Parse & Sanitise Code node, and the Respond to Webhook node on the right.

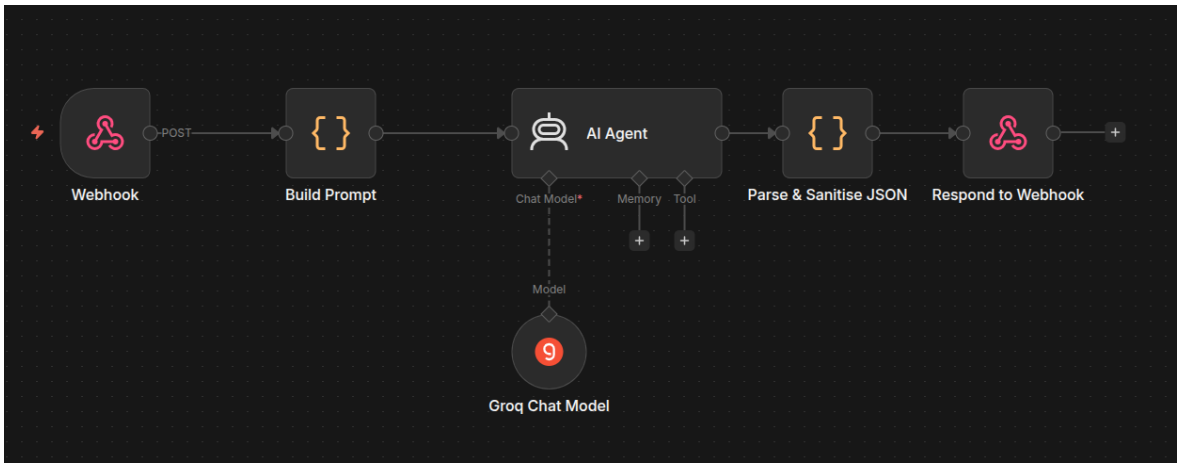

Figure 3.8: N8N workflow — AI storefront redesign

Figure 3.9 complements the workflow screenshot with the end-to-end interaction sequence between seller, backend, n8n and the LLM provider at the design level.

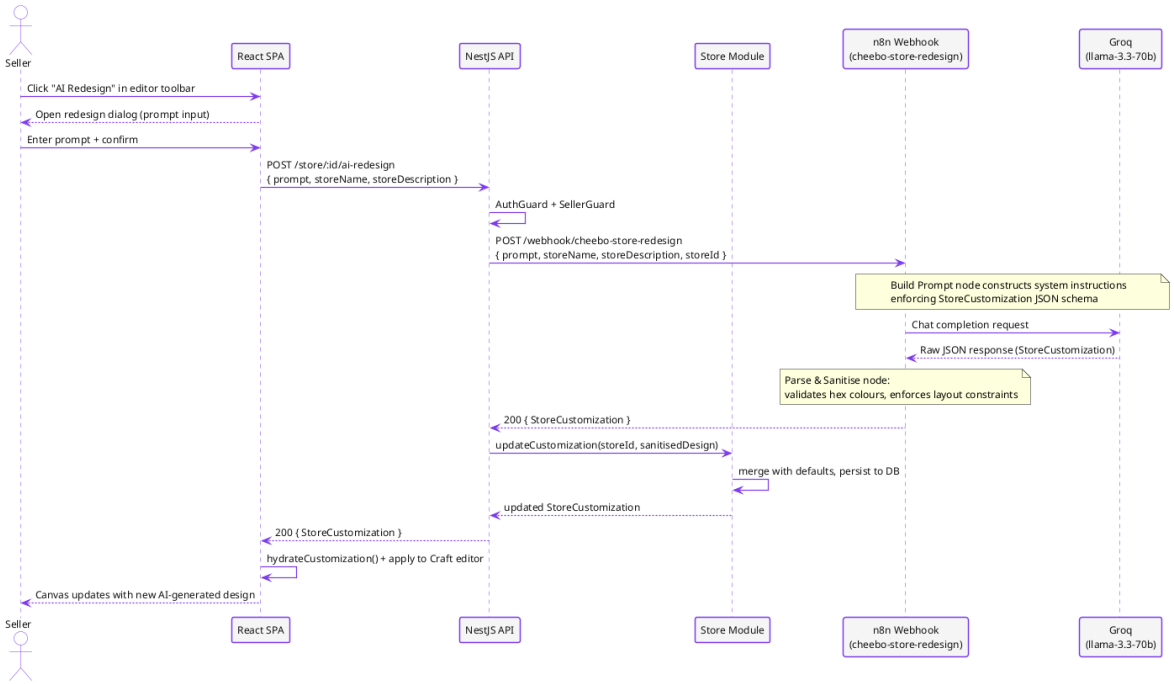

Figure 3.9: AI storefront redesign — interaction sequence

3.6 Class Diagram

Figure 3.10 presents the UML class diagram of the main domain entities. Where the ER diagram of Section 3.3.1 focused on tables, foreign keys and cardinalities, the class diagram emphasises the *behaviour* attached to each entity — the methods invoked by the service layer during use case execution — and the *enumerations* that constrain the state of every long-lived object.

Three structural relationships dominate the model:

- **User-driven aggregation.** The `User` class plays the role of the application's principal aggregate. It has a 1:0..1 association with `SellerApplication`, a 1:N composition with `Store` (one user, multiple stores on the premium tier), and a 1:N association with `Order` as a customer.
- **Store/Product composition.** `Store` composes `Product` (filled diamond): products cannot exist independently of their store, and cascading deletes are enforced at both the Prisma and database levels. The same composition links `Product` to its `ProductImage` children.
- **Order/OrderItems aggregation.** `Order` aggregates `OrderItems` (open diamond): order lines always belong to exactly one order, but the `Product` they reference has an independent lifecycle from the order. This is what allows historical orders to keep displaying a product that has since been deleted from a store.

Four enumerations (`UserRole`, `StoreStatus`, `OrderStatus`, `VerificationStatus`) constrain the state machines of their owning entities. Methods shown on each class correspond directly to service-layer operations exposed by the matching NestJS module — `Store.suspend(reason)` maps to `StoreService.suspend()`, `Product.computeSalePrice()` maps to the helper used by every product query, and so on.

Figure 3.10 renders the resulting model. Aggregate roots (`User`, `Store`, `Order`) appear with their associations and the multiplicities that govern them; supporting classes (`ProductImage`, `ProductTag`, `OrderItems`) sit alongside the entity they belong to; and the four enumerations are linked to their owning classes through dashed dependency arrows.

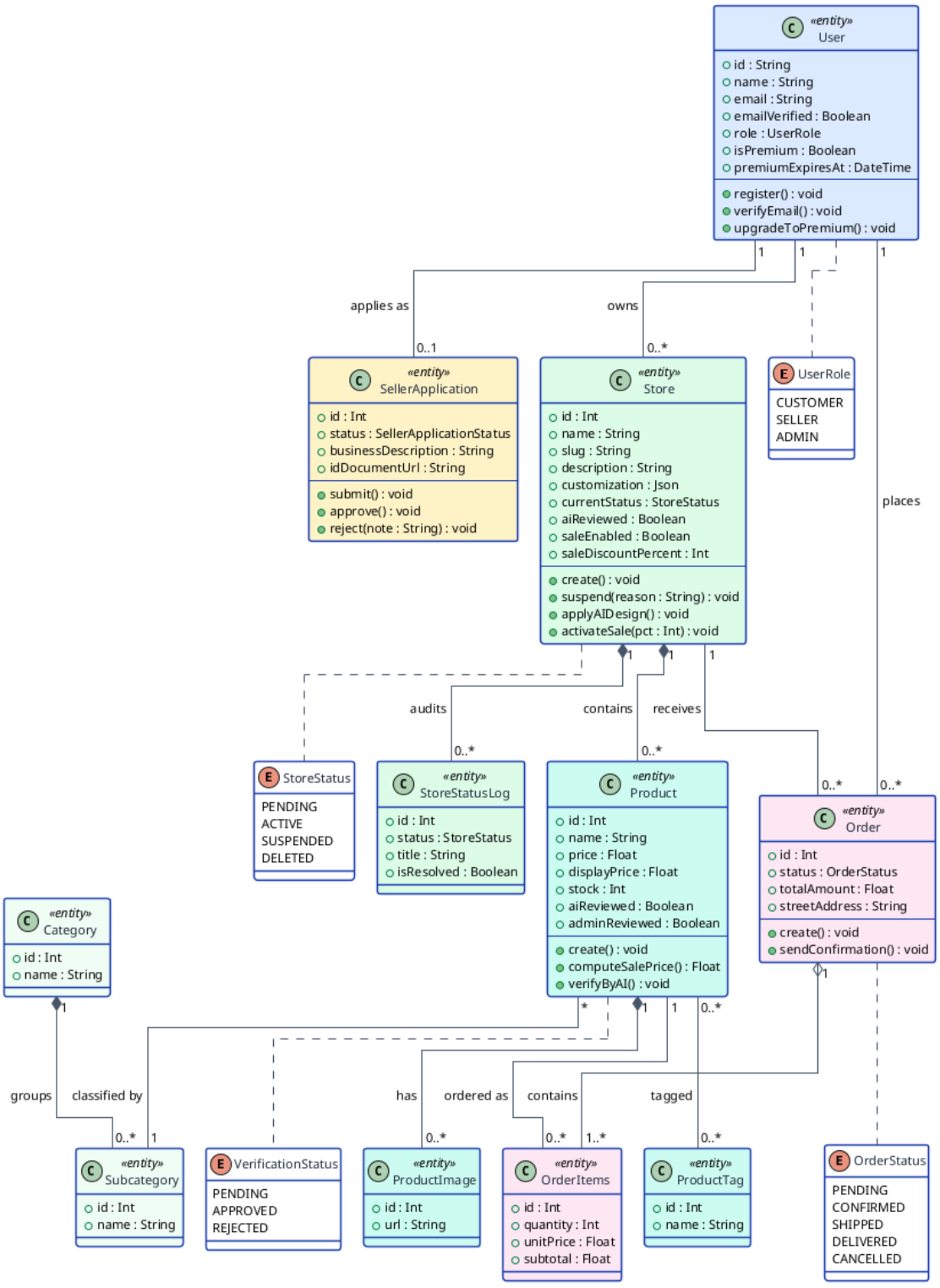

Figure 3.10: UML class diagram of the main domain entities

Key Takeaways

- The system is a three-tier architecture — React SPA, NestJS API, MariaDB —

with n8n as an external automation layer reached only through webhooks.

- The NestJS backend is composed of around twenty feature modules grouped into six functional clusters (cross-cutting, user, storefront, catalogue, commerce, AI), each mapping one-to-one to a functional requirement domain.
- The Prisma schema is split into eighteen files for reviewability, and uses the JSON `customization` column on `Store` to persist the Craft.js editor state without proliferating tables.
- The REST API follows resource-oriented design, validates every payload with `class-validator` DTOs and enforces RBAC through the `@Roles()` decorator.
- Three independent n8n workflows — verification, DB chat, storefront redesign — isolate prompt and provider choices from the backend.

Chapter Conclusion

This chapter translated the requirements of Chapter 2 into a concrete, implementable design. The three-tier layout established a clear boundary between presentation, business logic and data, while the n8n layer added an orthogonal AI dimension without polluting the core. The NestJS module catalogue, the React SPA layout, the Docker Compose topology and the Prisma schema were each presented at a level of detail that lets a reader *reconstruct* the system from the diagrams alone. The REST API design principles and the endpoint catalogue defined the contract between frontend and backend, and the three n8n workflows formalised the design of every AI feature. The class diagram in the final section bound all of these views together by showing the behavioural model that the next chapter actually implements.

Chapter 4 Development and Implementation

This chapter details the development environment, backend and frontend implementation decisions, and the AI integration via N8N.

4.1 Development Environment

4.1.1 Hardware and Software

4.1.2 VSCode and Extensions

4.1.3 pnpm Monorepo Structure

The project is organised as a `pnpm` workspace monorepo with three top-level packages. Figure 4.1 shows the directory tree: `backend/` contains the NestJS API, `frontend/` the React SPA, and `terraform/` the infrastructure-as-code definitions.

[IMAGE TO INSERT: Monorepo directory tree (backend/, frontend/, terraform/)]

Figure 4.1: Cheebo monorepo structure

4.2 Backend (NestJS)

4.2.1 Project Structure

4.2.2 Key Modules

4.2.3 Authentication Guards and Role-Based Access Control

Every protected route is secured through a combination of `AuthGuard` and the `@Roles()` decorator. Figure 4.2 shows a representative controller method annotated with both guards, illustrating how the framework enforces role-based access at the route level before the handler executes.

[IMAGE TO INSERT: Code snippet — AuthGuard + @Roles() decorator]

Figure 4.2: Role-based access control implementation

4.2.4 File Upload System (Multer)

4.2.5 Email System (Nodemailer)

4.3 Frontend (React + Vite)

4.3.1 Project Structure

4.3.2 Routing and Layouts

4.3.3 State Management (TanStack Query + Zustand)

4.3.4 Admin Dashboard Panels

Figure 4.3 shows the administrator dashboard, which centralises seller application reviews, store moderation, user management, and featured content curation in a single tabbed interface.

Figure 4.3: Admin dashboard

4.3.5 Store and Product Management Interface

Figure 4.4 presents the seller-facing store and product management panels, from which a seller can create and edit their store, manage their product catalogue, upload images, and monitor incoming orders.

[IMAGE TO INSERT: Screenshot — Store / product management (seller view)]

Figure 4.4: Store and product management interface

4.4 AI Integration with N8N

4.4.1 AI Product/Store Verification Pipeline

4.4.2 Database Chat Assistant

4.4.3 N8N Workflow — AI Storefront Redesign

The AI Storefront Redesign workflow allows a seller to regenerate their entire store visual identity by providing a short natural-language prompt from inside the Craft.js editor. The sequence is illustrated in Figure 4.5.

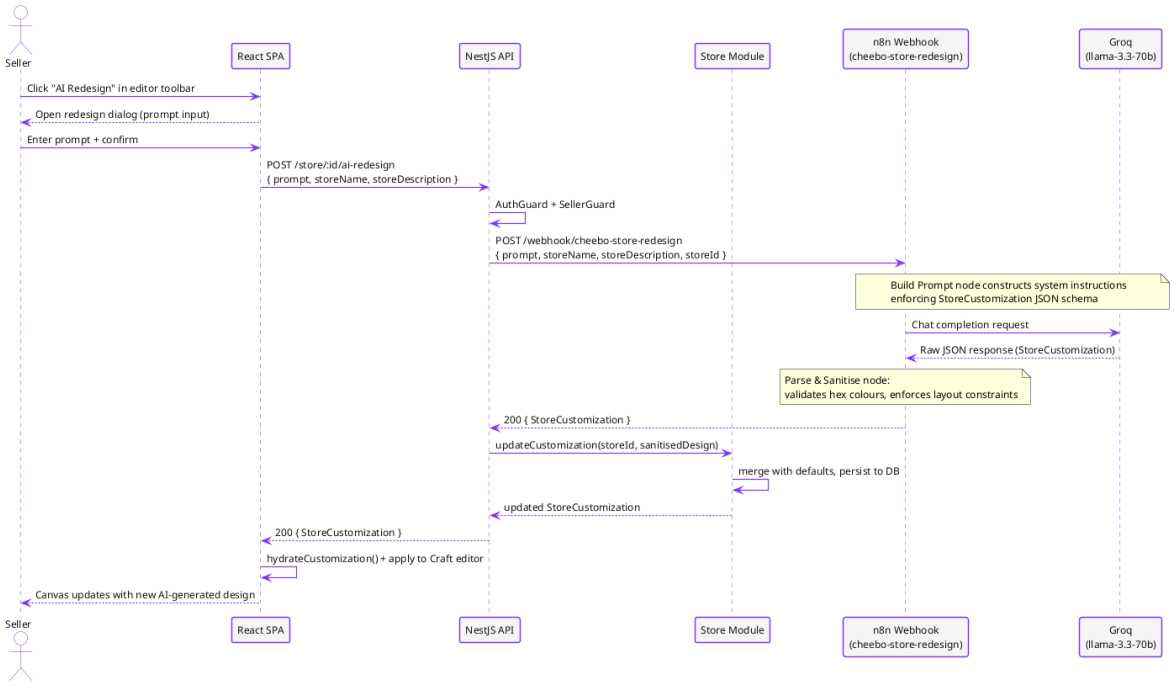

Figure 4.5: Sequence diagram — AI storefront redesign workflow

The seller clicks the *AI Redesign* button in the editor toolbar, which opens a dialog prompting for a short description of the desired aesthetic (e.g. *"dark luxury jewellery brand with gold accents"*). The React frontend calls `POST /store/:id/ai-redesign`, passing the prompt alongside the store name and description for context.

On the backend, the NestJS Store Module forwards the request to the **cheebo-store-redesign** n8n webhook. The workflow consists of three nodes:

1. **Build Prompt** — a Code node that constructs a system-level instruction enforcing a strict JSON schema matching the `StoreCustomization` type. The schema covers every configurable section: theme colours and fonts, announcement bar, banner, header, product grid, sidebar, and footer.

2. **AI Agent** — an AI Agent node connected to the Groq `llama-3.3-70b-versatile` model. The agent receives the assembled system prompt and the seller's free-text request, then returns a JSON object conforming to the `StoreCustomization` schema.

3. **Parse & Sanitise** — a Code node that parses the raw LLM output, validates all hex colour values with a regex, clamps numeric fields to valid ranges, and falls back to safe defaults for any field that is missing or malformed.

The sanitised `StoreCustomization` object is returned to the NestJS backend, which merges it with the store defaults and persists it through the existing `updateCustomization` path. The frontend then calls `hydrateCustomization()` on the response and applies

the result directly to the Craft.js editor, giving the seller an immediate live preview of the AI-generated design without a page reload.

4.4.4 Google Gemini / Groq Integration

4.4.5 N8N Workflow Implementation

4.5 Interface Screenshots

This section presents screenshots of the deployed platform, grouped by the three user roles (customer, seller, administrator) and a final group covering the AI assistant.

4.5.1 Customer Experience

Figure 4.6 shows the Cheebo public landing page, which surfaces featured stores, featured products, and the tag-based recommendation section. The navigation bar provides direct access to the catalogue, login, and cart.

Figure 4.6: Landing page

Figure 4.7 shows the global stores listing page, with the search bar, store cards, and the active-sale badges displayed on stores currently running a promotion.

Figure 4.7: Stores listing page

Figure 4.8 shows the category-filtered products browse page, with sidebar filters (category, sub-category, price range, sort) and the product grid displaying discounted prices on products belonging to stores that have activated a sale.

Figure 4.8: Product browse page

Figure 4.9 shows the product detail page, with the image carousel, description, tags, stock indicator, sale price (when applicable), and the *Add to cart* action.

[IMAGE TO INSERT: Screenshot — Product detail page with sale price and add-to-cart]

Figure 4.9: Product detail page

Figure 4.10 shows the global `Ctrl+K` search dialog (`cmdk`), which surfaces both products and stores in real time as the customer types.

[IMAGE TO INSERT: Screenshot — Ctrl+K command palette showing product and store results]

Figure 4.10: Global search dialog

Figure 4.11 shows the checkout page with the multi-store cart summary, shipping

form, per-store subtotal lines, and the order total reflecting active sale discounts.

[IMAGE TO INSERT: Screenshot — Checkout page with multi-store cart and shipping form]

Figure 4.11: Checkout page

4.5.2 Seller Dashboard

Figure 4.12 presents the seller store dashboard, from which a seller can monitor live store metrics, access the customisation editor, and review recent order activity.

[IMAGE TO INSERT: Screenshot — Seller store dashboard with metrics and quick actions]

Figure 4.12: Store dashboard

Figure 4.13 shows the Craft.js WYSIWYG storefront editor, with the sections panel on the left, the live canvas in the centre, and the settings panel on the right. Selecting any section on the canvas opens its inspector.

Figure 4.13: WYSIWYG storefront editor

Figure 4.14 shows the AI Redesign dialog, in which a seller submits a natural-language prompt to regenerate the storefront design end-to-end.

Figure 4.14: AI Redesign dialog

Figure 4.15 shows the Sale configuration panel, where a seller toggles the store-wide sale, sets the discount percentage, custom label, and optional expiry date.

[IMAGE TO INSERT: Screenshot — Sale panel with percentage, label and expiry controls]

Figure 4.15: Store-wide sale configuration panel

Figure 4.16 shows the seller's product management panel with the list view, create/edit form, image upload, and tag selector.

[IMAGE TO INSERT: Screenshot — Seller product management (list + create/edit form)]

Figure 4.16: Product management

Figure 4.17 shows the per-store analytics page available to sellers, displaying revenue trends, top-selling products, order volumes, and visitor statistics.

[IMAGE TO INSERT: Screenshot — Seller analytics page (revenue chart, top products)]

Figure 4.17: Analytics page

4.5.3 Administrator Tools

Figure 4.18 shows the admin dashboard landing page, with platform-wide metrics, the navigation to every admin panel, and the recent activity feed.

[IMAGE TO INSERT: Screenshot — Admin dashboard overview with metrics and navigation]

Figure 4.18: Admin dashboard overview

Figure 4.19 shows the seller applications review panel, where the admin reviews each application's profile, store proposal, and approves or rejects with a moderation note.

Figure 4.19: Seller applications panel

Figure 4.20 illustrates the AI verification panel available to administrators. Each entry displays the entity name, the AI verdict, the LLM rationale, and override controls allowing the administrator to confirm or reverse the automated decision.

[IMAGE TO INSERT: Screenshot — AI verification panel with verdict, rationale, override controls]

Figure 4.20: AI verification panel

Figure 4.21 shows the users management panel, where the admin views every registered user, filters by role, and changes role assignments.

[IMAGE TO INSERT: Screenshot — Users management panel (list + role assignment)]

Figure 4.21: Users panel

Figure 4.22 shows the stores moderation panel, where the admin can suspend, reactivate, or cascade-delete a store with an attached reason logged to `StoreStatusLog`.

[IMAGE TO INSERT: Screenshot — Stores moderation panel (suspend / reactivate / delete actions)]

Figure 4.22: Stores moderation panel

Figure 4.23 shows the categories and sub-categories management panel, with create, rename, reorder, and delete actions.

[IMAGE TO INSERT: Screenshot — Categories and sub-categories management panel]

Figure 4.23: Categories panel

Figure 4.24 shows the home-page curation panel, where the admin selects the featured stores and products that appear on the landing page hero.

[IMAGE TO INSERT: Screenshot — Home-page curation panel (featured stores and products)]

Figure 4.24: Featured content curation panel

4.5.4 AI Assistant

Figure 4.25 shows the floating AI Chat assistant (`AiChatFab`), a docked button on the administrator dashboard that opens a conversational panel where the administrator can query the database in natural language (orders, users, products, stores, verification queues) for operational insights.

[IMAGE TO INSERT: Screenshot — Floating AI chat assistant (FAB + open panel)]

Figure 4.25: AI chat assistant

Chapter 5 Infrastructure and Deployment

This chapter covers Docker containerisation, infrastructure as code with Terraform, the GitHub Actions CI/CD pipeline, DNS management, and the production environment.

5.1 Containerisation with Docker

5.1.1 Dockerfiles (Frontend and Backend)

5.1.2 Docker Compose Configuration

5.1.3 Multi-Service Networking

All services communicate over a private Docker bridge network, meaning the backend, database, and n8n are never directly exposed to the public internet. Only the frontend container publishes port 80. Figure 5.1 shows the network topology and how Nginx on the frontend container proxies all API paths to the backend service over the internal network.

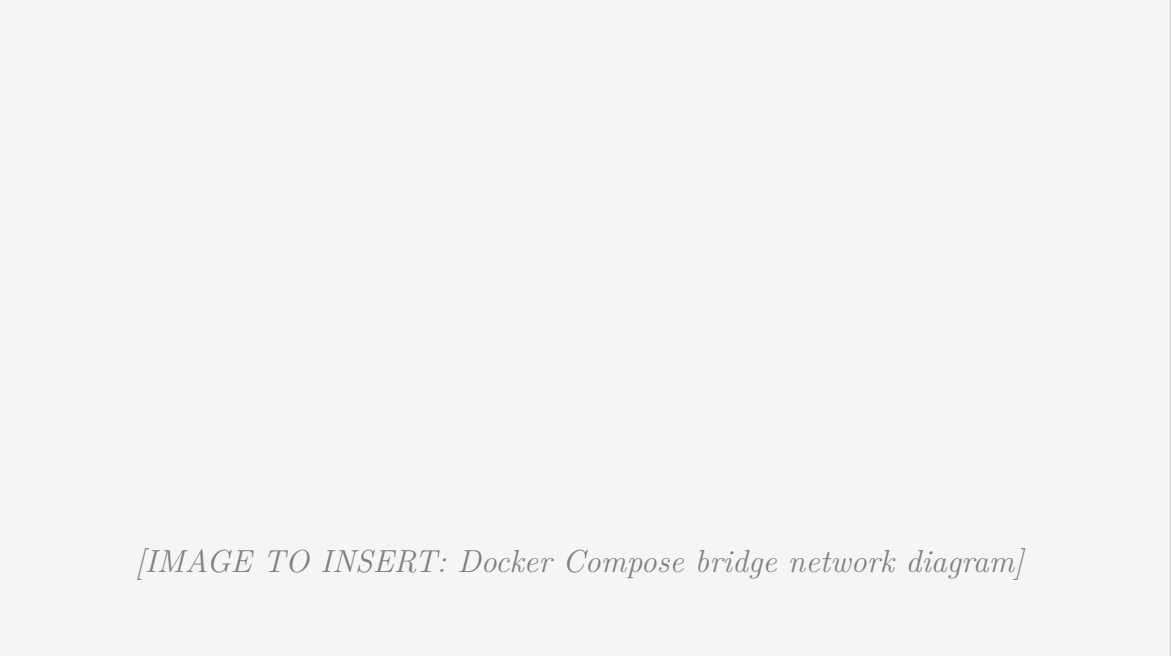

Figure 5.1: Docker Compose network architecture

5.2 Infrastructure as Code with Terraform

5.2.1 DigitalOcean Provider

5.2.2 VPC, Droplet and Firewall Resources

5.2.3 DNS Management with DigitalOcean

Rather than managing DNS records at the domain registrar (name.com) through a separate API, the project delegates DNS entirely to DigitalOcean using the same Terraform provider that manages the droplet. Two resources are declared in `main.tf`:

- `digitalocean_domain` — creates the DNS zone for `nadimjebali.engineer` on DigitalOcean's authoritative nameservers and automatically adds the apex (`@`) A record pointing to the droplet's IPv4 address.
- `digitalocean_record` — adds a second A record for the `www` subdomain, also pointing to the droplet.

Both records reference `digitalocean_droplet.cheebo.ipv4_address` directly. This means that if the droplet is ever replaced (e.g. after a destructive Terraform change), Terraform updates both DNS records in the same `apply` run that creates the new droplet, with no manual intervention required.

The domain's nameservers are delegated to DigitalOcean (`ns1-ns3.digitalocean.com`) from the name.com registrar panel, which is a one-time configuration step. After that, A-record updates propagate in under five minutes thanks to a 300-second TTL.

> **Key Takeaways**
>
> DNS is version-controlled alongside infrastructure — the domain always points to the current droplet IP with no manual update step.
>
> TTL 300 s means a full redeploy (destroy old droplet, create new one, update DNS) converges in under ten minutes end-to-end.

5.2.4 Remote State Management with DigitalOcean Spaces

Terraform state is stored remotely in a DigitalOcean Spaces bucket configured as an S3-compatible backend. This ensures that every CI/CD run reads from and writes to a single shared state file, preventing conflicts if the workflow is triggered concurrently. Figure 5.2 shows the key resource declarations in `main.tf`, including the VPC, the droplet, and the DNS records.

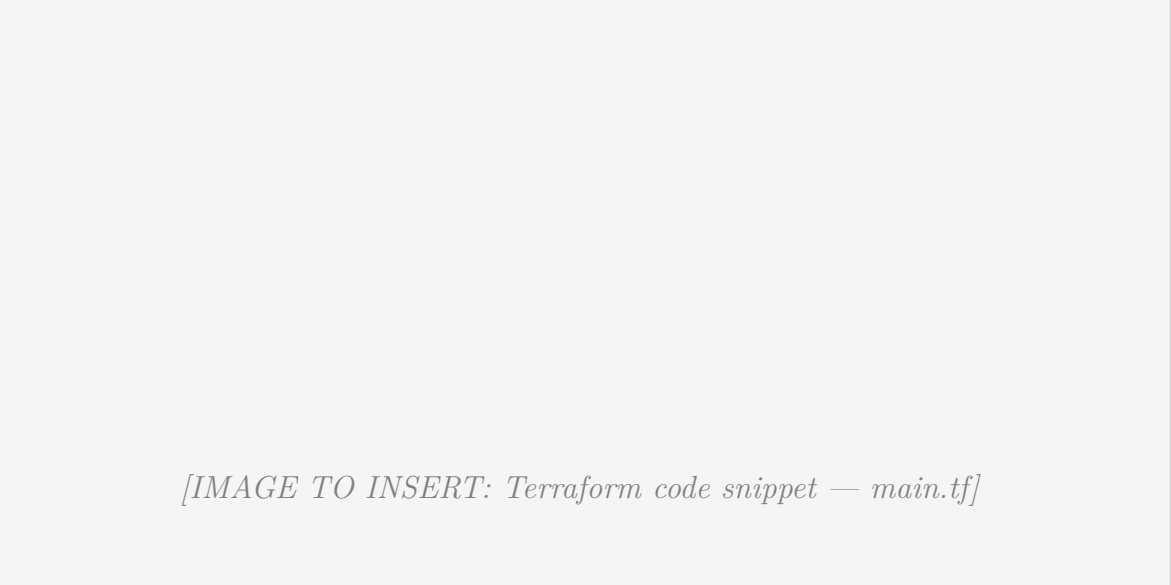

Figure 5.2: Terraform resources

5.3 CI/CD Pipeline with GitHub Actions

5.3.1 Workflow Overview

The CI/CD pipeline is defined as a single GitHub Actions workflow file that runs on every push to the `main` branch. Figure 5.3 presents the three parallel jobs — *provision* (Terraform), *build-backend*, and *build-frontend* — followed by the *deploy* job that SSH-deploys the composed stack once all three complete.

Figure 5.3: GitHub Actions CI/CD pipeline

5.3.2 Build Stage (Docker Build and Push to Docker Hub)

5.3.3 Deployment Stage (SSH and Docker Compose)

5.3.4 Environment Secrets Management

5.4 Production Environment

5.4.1 DigitalOcean Droplet Specifications

The production workload runs on a single DigitalOcean droplet provisioned by Terraform with the following specifications:

Property	**Value**
Region	Frankfurt (fra1)
Size	s-2vcpu-2gb (2 vCPUs, 2 GB RAM)
Image	Ubuntu 24.04 LTS
VPC	cheebo-vpc (192.168.1.0/24)
Public URL	`http://nadimjebali.engineer`
DNS	Managed by DigitalOcean (Terraform)

The application is accessed exclusively through the domain name `nadimjabali.engineer`. The raw droplet IP is reserved for SSH access by the CI/CD pipeline and is never exposed to end users. CORS on the backend is configured with the domain as the single trusted origin, so any attempt to reach the API directly from the IP is rejected by the browser's same-origin policy.

5.4.2 Production Service Architecture

Figure 5.4 depicts the production environment as deployed on the DigitalOcean droplet. All four Docker Compose services run on the same host inside a private bridge network; only the frontend Nginx container is reachable from the public internet via the domain `nadimjebali.engineer`.

Figure 5.4: Production architecture

5.4.3 N8N Deployment Alongside the Application

General Conclusion

Summary of Achievements

Implemented Features vs Initially Planned Features

Challenges Encountered and Solutions Provided

Current Implementation Limitations

Future Improvements

Bibliography and Webography

- NestJS Official Documentation — https://docs.nestjs.com
- React Official Documentation — https://react.dev
- Prisma ORM Documentation — https://www.prisma.io/docs
- TanStack Query Documentation — https://tanstack.com/query
- Docker Documentation — https://docs.docker.com
- Terraform Documentation — https://developer.hashicorp.com/terraform
- N8N Documentation — https://docs.n8n.io
- GitHub Actions Documentation — https://docs.github.com/en/actions
- DigitalOcean Developer Docs — https://docs.digitalocean.com

Chapter A Complete Prisma Schema

Listing A.1: Prisma schema — excerpt

```
% TODO: paste the contents of backend/prisma/schema/ here
```

Chapter B API Endpoints Reference

Chapter C N8N Workflow JSON Exports

Listing C.1: N8N workflow — AI verification (JSON)

```
% TODO: paste the JSON exported from N8N
```

Chapter D Docker Compose File

Listing D.1: docker-compose.yml

```
% TODO: paste the contents of docker-compose.yml
```

Chapter E GitHub Actions Workflow

Listing E.1: .github/workflows/deploy.yml

```
% TODO: paste the CI/CD workflow content
```